

universidad
de león

Departamento de Ingenierías
Mecánica, Informática y Aeroespacial

MÁSTER UNIVERSITARIO EN ROBÓTICA E INTELIGENCIA ARTIFICIAL

Trabajo de Fin de Máster

Aprendizaje por Refuerzo Multiagente para
Conducción Autónoma en CARLA Bajo Lluvia
Intensa

Multi-Agent Reinforcement Learning for
Autonomous Driving in CARLA under Heavy Rain

Autor: Jose Javier González García-Mascaraque

Tutor: Miguel Ángel González Santamarta

UNIVERSIDAD DE LEÓN Departamento de Ingenierías Mecánica, Informática y Aeroespacial MÁSTER UNIVERSITARIO EN ROBÓTICA E INTELIGENCIA ARTIFICIAL Trabajo de Fin de Máster	
ALUMNO: Jose Javier González García-Mascaraque	
TUTOR: Miguel Ángel González Santamarta	
TÍTULO: Aprendizaje por Refuerzo Multiagente para Conducción Autónoma en CARLA bajo Lluvia Intensa	
TITLE: Multi-Agent Reinforcement Learning for Autonomous Driving in CARLA under Heavy Rain	
CONVOCATORIA: Septiembre, 2026	
RESUMEN: Con el objetivo de garantizar una navegación segura y coordinada en entornos complejos, se diseña, desarrolla y evalúa un sistema de conducción autónoma multiagente utilizando técnicas de aprendizaje por refuerzo en el simulador CARLA bajo condiciones meteorológicas de lluvia intensa. La metodología se basa en la implementación de MAPPO que sigue un esquema de entrenamiento centralizado y ejecución descentralizada. Se analiza la inclusión de varios sensores para la toma de decisiones y prevención de colisiones como la segmentación semántica, la cámara RGB o el radar. También con un vector de estado extraído del simulador, actuando como un Ground Truth. Se desarrolla en el mapa Town10 de CARLA, comparando la convergencia del sistema en entornos dinámicos al incluir vehículos no jugables(NPCs) o simplemente los agentes a entrenar, siempre con fricción reducida por la lluvia y las acumulaciones de agua. Los resultados demuestran la eficiencia de la política aprendida para coordinar los agentes y seguir una ruta manteniendo una conducción equilibrada. Se aprecian también la alta sensibilidad de MAPPO al ruido de observación. Aporta así un análisis sobre el desarrollo de políticas en sistemas multiagentes en condiciones climáticas extremas.	
Palabras clave: Aprendizaje por Refuerzo Multiagente, conducción autónoma, MAPPO, CARLA, Ground Truth, condiciones meteorológicas adversas	
Firma del alumno:	VºBº Tutor:

UNIVERSITY OF LEÓN Department of Mechanical, Computer and Aerospace Engineering MASTER'S DEGREE IN ROBOTICS AND ARTIFICIAL INTELLIGENCE Master's Thesis	
STUDENT: Jose Javier González García-Mascaraque	
TUTOR: Miguel Ángel González Santamarta	
TÍTULO: Aprendizaje por Refuerzo Multiagente para Conducción Autónoma en CARLA bajo Lluvia Intensa	
TITLE: Multi-Agent Reinforcement Learning for Autonomous Driving in CARLA under Heavy Rain	
CALL: September, 2026	
ABSTRACT: With the objective of guaranteeing safe and coordinated navigation in complex environments, a multi-agent autonomous driving system is designed, developed, and evaluated using reinforcement learning techniques in the CARLA simulator under heavy rain weather conditions. The methodology is based on the implementation of MAPPO, which follows a centralized training and decentralized execution scheme. The inclusion of several sensors for decision-making and collision prevention is analyzed, such as semantic segmentation, RGB camera, or radar. Also with a state vector extracted from the simulator, acting as Ground Truth. It is developed in the Town10 map of CARLA, comparing the system's convergence in dynamic environments by including non-player vehicles (NPCs) or simply the agents to be trained, always with reduced friction due to rain and water accumulations. The results demonstrate the efficiency of the learned policy to coordinate the agents and follow a route while maintaining balanced driving. High sensitivity of MAPPO to observation noise is also observed. It thus contributes an analysis on policy development in multi-agent systems under extreme weather conditions.	
Key words: Multi-Agent Reinforcement Learning, autonomous driving, MAPPO, CARLA, Ground Truth, adverse weather conditions	
Student signature:	Advisor Approval:

Índice general

Índice de figuras	VI
Índice de tablas	VII
Glosario de términos	IX
Introducción	1
1. Marco teórico	4
1.1. Fundamentos del Reinforcement Learning	4
1.2. Paradigmas de aprendizaje	6
1.3. Deep Reinforcement Learning	7
1.4. Principales Algoritmos	7
1.4.1. Algoritmos basados en funciones de valor (Value-Based)	8
1.4.2. Algoritmos basados en políticas (Policy-Based)	8
1.4.3. Algoritmos Actor-Critic	9
1.5. Proximal Policy Optimization (PPO)	9
1.6. Multi-Agent Deep Reinforcement Learning	10
1.6.1. Multi-Agent Proximal Policy Optimization (MAPPO)	12
2. Estado del arte	13
2.1. Aprendizaje por refuerzo aplicado a la conducción autónoma	13
2.2. Aprendizaje por refuerzo multiagente en conducción autónoma	14
2.3. Problemas en Condiciones Meteorológicas Adversas	15
2.4. Análisis del estado del arte actual	16
3. Gestión de proyecto software	17
3.1. Alcance del proyecto	17
3.1.1. Definición del proyecto	17
3.2. Metodología de trabajo	18

3.2.1.	Identificación de tareas	19
3.2.2.	Estimación de tareas	20
3.3.	Plan de trabajo	21
3.4.	Gestión de recursos	22
3.4.1.	Presupuesto	22
3.5.	Gestión de riesgos	25
3.5.1.	Identificación de riesgos	25
3.6.	Legislación y normativa	27
4.	Metodología	29
4.1.	Selección del entorno de simulación	29
4.2.	Configuración del entorno de simulación	31
4.3.	Formulación del problema como Dec-POMDP	34
4.4.	Observaciones	36
4.4.1.	Vector de estado	36
4.4.2.	Evolución de la Representación Perceptual y Abstracción Ground Truth	37
4.5.	Espacio de acciones	39
4.6.	Función de recompensa	40
4.7.	Aprendizaje en el entrenamiento	42
4.7.1.	Recopilación de experiencias	42
4.7.2.	Estimación de las ventajas	43
4.7.3.	Actualización de las políticas	44
4.7.4.	Actualización del crítico	45
4.7.5.	Proceso de actualización	46
5.	Solución	47
5.1.	Descripción de la solución	47
5.2.	El proceso de desarrollo	48
5.2.1.	Análisis	48
5.2.2.	Diseño e Implementación	49
6.	Evaluación	55
6.1.	Proceso de entrenamiento	55
6.1.1.	Definición de métricas y limitaciones	55
6.1.2.	Estabilidad de la optimización	56
6.1.3.	Evolución de la recompensa	57
6.1.4.	Progreso hacia el destino	58

6.1.5. Seguridad de conducción	59
6.1.6. Síntesis cuantitativa	60
6.1.7. Asimetría entre agentes y Limitaciones	61
6.2. Proceso de evaluación	61
6.2.1. Rendimiento con percepción idealizada de CARLA	62
6.2.2. Ablación del stack de percepción	64
6.2.3. El embudo de la detección	66
6.2.4. Separabilidad de la señal de radar	66
Conclusión	68
Lista de referencias	71
A. Control de versiones	75
B. Seguimiento de proyecto fin de máster	76
B.1. Forma de seguimiento	76
B.2. Planificación inicial	77
B.3. Planificación final	77
C. Declaración de uso de Inteligencia Artificial	79

Índice de figuras

1.1. RL	4
1.2. algoritmos DRL	8
1.3. Diferencia CTDE-DTDE	12
3.1. Diagrama de Gantt.	21
4.1. Town10HD	32
6.1. Perdidas	56
6.2. Recompensa	57
6.3. Ruta completada	58
6.4. Llegadas	58
6.5. colisiones	59
6.6. velocidad	60
6.7. Desenlaces	63
6.8. Ruta completada	64
6.9. Ruta completada rada	65
6.10. Velocidad radial	67

Índice de tablas

3.1. Estimación de tareas.	19
3.2. Planificación y estimación de tareas por Sprints.	20
3.3. Presupuesto estimado para los recursos humanos.	22
3.4. Coste estimado de la infraestructura computacional.	23
3.5. Presupuesto estimado para software y licencias.	23
3.6. Presupuesto final estimado del proyecto.	25
3.7. Análisis de riesgos del proyecto.	26
4.1. Comparativa de simuladores utilizados en aprendizaje por refuerzo para conducción autónoma.	29
4.2. Configuración del entorno de simulación en CARLA.	33
4.3. Variables incluidas en el vector de estado de observación del agente. .	38
4.4. Recompensas, penalizaciones y condiciones de finalización del episodio.	42
5.1. Evolución iterativa del sistema de percepción y justificación de deci- siones.	51
5.2. Composición del vector de observación local o_i (18 variables).	52
6.1. Evolución del entrenamiento en ventanas de episodios (2048 <i>steps</i> con <i>respawn</i>). Intervalo de confianza al 95 % (Poisson).	60
6.2. Resultados de evaluación por escenario y agente, con acciones de- terministas (media de la gaussiana). Cada episodio aporta un único desenlace.	62
6.3. Ablación del <i>stack</i> de percepción en el escenario con tráfico. La po- lítica y los pesos son idénticos en las tres filas: solo cambia el origen de las <i>features</i> . Las columnas Δ representan la diferencia frente al <i>ground truth</i>	64

6.4.	Embudo de procesamiento del radar por <i>tick</i> . El <i>clustering</i> agrupa las múltiples reflexiones de un mismo objeto; la validación contra <i>ground truth</i> descarta las que no corresponden a ningún actor. El porcentaje final indica cuánta de la señal cruda es entorno estático.	66
6.5.	Separabilidad por velocidad radial calculada de forma independiente en cada condición. La velocidad radial de un objeto estático coincide con la del propio vehículo. Agregar ambas poblaciones cancela sus sesgos y produce un valor espurio.	66
B.1.	Comparativa de dedicación por Sprints entre la planificación inicial y la final.	78

Glosario de términos

Actor : Componente en la arquitectura Actor-Critic encargada de parametrizar la política y seleccionar las acciones del agente en el entorno.

Actor-Critic : Arquitectura de Aprendizaje por Refuerzo que combina métodos basados en políticas (Actor) con la estimación de funciones de valor (Crítico).

CARLA (*Car Learning to Act*): Simulador de código abierto para la investigación y desarrollo de sistemas de conducción autónoma en entornos urbanos.

CBF (*Control Barrier Function*): Herramienta matemática utilizada en control de sistemas para garantizar restricciones de seguridad definiendo conjuntos invariantes.

CNN (*Convolutional Neural Network*): Arquitectura de red neuronal profunda especializada en el procesamiento de imágenes, mediante el uso de filtros convolucionales para extraer características visuales de forma jerárquica.

CTDE (*Centralized Training with Decentralized Execution*): Paradigma de MARL donde los agentes comparten información durante el entrenamiento pero ejecutan acciones de forma independiente.

Dec-POMDP (*Decentralized Partially Observable Markov Decision Process*): Formulación matemática para modelar decisiones secuenciales en entornos multi-agente con información parcial.

DRL (*Deep Reinforcement Learning*): Combinación de Aprendizaje por Refuerzo con redes neuronales profundas para aproximar políticas o funciones de valor.

DTDE (*Decentralized Training with Decentralized Execution*): Paradigma en MARL donde tanto el proceso de entrenamiento como la ejecución de acciones son totalmente independientes para cada agente.

- GAE** (*Generalized Advantage Estimation*): Algoritmo para estimar la función de ventaja en RL equilibrando el sesgo y la varianza en los gradientes de la política.
- IL** (*Imitation Learning*): Técnica de aprendizaje donde el agente aprende a resolver una tarea imitando las demostraciones de un experto (humano o algoritmo heurístico).
- IPPO** (*Independent Proximal Policy Optimization*): Extensión de PPO a entornos multiagente donde cada agente se entrena de forma independiente considerando a los demás como parte del entorno.
- LiDAR** (*Light Detection and Ranging*): Sensor que emite pulsos de luz láser para medir distancias y construir representaciones 3D en forma de nube de puntos.
- MAPPO** (*Multi-Agent Proximal Policy Optimization*). Adaptación del algoritmo PPO para problemas multiagente, comúnmente bajo la arquitectura CTDE.
- MARL** (*Multi-Agent Reinforcement Learning*): Subcampo del RL centrado en entornos donde interactúan múltiples agentes coordinados, competitivos o mixtos.
- MDP** (*Markov Decision Process*): Formulación matemática para modelar problemas de toma de decisiones secuenciales donde las transiciones entre estados cumplen la propiedad de Markov.
- MPC** (*Model Predictive Control*): Algoritmo de control avanzado que utiliza un modelo dinámico del sistema para optimizar las acciones dentro de un horizonte temporal futuro.
- NPC** (*Non-Player Character*): Vehículos o peatones del simulador controlados por la IA del entorno (tráfico de fondo) que no forman parte del aprendizaje.
- PEM** (*Presupuesto de Ejecución Material*): Importe resultante de la suma de los costes directos e indirectos asociados a la ejecución física del proyecto, utilizado como base imponible para calcular los gastos generales, el beneficio industrial y los impuestos.
- Política** (π) : Función que mapea los estados o las observaciones recibidas por el agente hacia la distribución de acciones que debe tomar.
- PPO** (*Proximal Policy Optimization*): Algoritmo de RL basado en gradientes de política que acota las actualizaciones del modelo para mantener la estabilidad del entrenamiento.

RL (*Reinforcement Learning*): Rama del aprendizaje automático donde un agente aprende a tomar decisiones secuenciales mediante interacción por ensayo y error con un entorno.

ROS/ROS2 (*Robot Operating System*): Middleware de código abierto estructurado en nodos interconectados que proporciona abstracción de hardware.

Segmentación semántica : Técnica de visión por computador que asigna una etiqueta de clase (ej. vehículo, calzada, peatón) a cada píxel individual de una imagen.

V2V (*Vehicle-to-Vehicle*): Tecnología de comunicación inalámbrica que permite el intercambio directo de información de telemetría entre vehículos.

VAE (*Variational Autoencoder*): Modelo generativo profundo que codifica entradas en un espacio latente probabilístico continuo para reconstruir datos.

Waypoint : Punto espacial de referencia con coordenadas y orientación definidas dentro de un mapa, utilizado por los algoritmos de navegación para guiar la trayectoria y evaluar el progreso del vehículo.

Introducción

La conducción autónoma constituye uno de los principales retos actuales dentro de la inteligencia artificial y la robótica. El desarrollo de vehículos capaces de percibir el entorno, planificar y actuar de forma segura puede aumentar la seguridad vial, mejorando la eficiencia del tráfico. Sin embargo, esto requiere resolver problemas relacionados entre otros con percepción o planificación de trayectorias en escenarios dinámicos [1, 2].

Las aproximaciones tradicionales abordadas usando arquitecturas modulares basadas en reglas demuestran un gran rendimiento en entornos controlados pero presentan dificultades para cubrir la gran variedad de situaciones a las que se enfrentan. Por ello, técnicas que generalizan mejor como IL(*Imitation Learning*) o RL(*Reinforcement Learning*) toman peso, al permitir abordar esto como un proceso de toma de decisiones secuencial, del cual aprender una política de comportamiento mientras se interactúa con el entorno [1, 3].

Además, la realidad del tráfico urbano exige la transición, inherentemente, hacia el Multi-Agent Reinforcement Learning, donde los vehículos aprendan políticas de comportamiento cooperativas para coordinar maniobras sin necesidad de un controlador centralizado durante la ejecución [2, 3].

Por otro lado, se encuentra el impacto de las condiciones climatológicas, que en caso de ser desfavorables aumenta la incertidumbre del entorno alterando tanto la dinámica del vehículo como la percepción sensorial [1].

En este contexto, el estudio se centrará en la aplicación de MAPPO sobre el simulador CARLA [4] para analizar múltiples vehículos autónomos cooperando por conseguir una circulación lo más óptima posible en condiciones de lluvia. El principal objetivo por tanto será evaluar la capacidad de este algoritmo de aprender una conducción segura y coordinada.

Planteamiento del problema

La conducción autónoma usando aprendizaje por refuerzo ha demostrado resultados prometedores en tareas como el seguimiento de carril y la evitación de obstáculos. Sin embargo, la mayor parte de estos trabajos consideran un único vehículo que actúa de forma aislada y en escenarios simplificados que no representan la complejidad del tráfico real [5].

En un entorno en el que múltiples vehículos deben compartir el espacio de circulación e interactuar, por afectar las decisiones de un agente al resto, pasa esto a tratarse como un problema inherentemente multiagente, donde es necesario desarrollar políticas para coordinar las acciones de varios vehículos [3].

A esta dificultad se suma la influencia de las condiciones meteorológicas adversas, en este caso la lluvia. Ésta modifica las condiciones de conducción y degrada la información de los sensores, incrementando la incertidumbre durante la percepción del entorno y la toma de decisiones [6]. Aunque simuladores como el que se tratará en este estudio, CARLA [4], permiten reproducir dichas situaciones, la literatura revisada muestra que generalmente los trabajos se centran en escenarios con condiciones meteorológicas favorables o en algoritmos de aprendizaje por refuerzo para un único agente, no combinando ambas facetas en apenas investigaciones. Asimismo, las investigaciones sobre aprendizaje por refuerzo multiagente se evalúan mayormente en simuladores que no reproducen con suficiente fidelidad la dinámica de la conducción urbana.

Como consecuencia, existe una oportunidad interesante en cuanto al estudio del comportamiento de sistemas de Multi-Agent Reinforcement Learning bajo condiciones meteorológicas adversas utilizando simuladores de alta fidelidad. En este contexto, el presente Trabajo de Fin de Máster propone implementar y evaluar un sistema basado en Multi-Agent Proximal Policy Optimization (MAPPO) sobre el simulador CARLA, con el objetivo de analizar la capacidad de coordinación y adaptación de múltiples vehículos autónomos cuando operan bajo condiciones de lluvia.

Objetivos

En este Trabajo de Fin de Máster, se plantean una serie de objetivos interconectados que tienen como finalidad principal:

El objetivo de este Trabajo de Fin de Máster consiste en diseñar, implementar y evaluar un marco de trabajo de Multi-Agent Reinforcement Learning basado en

MAPPO en el simulador CARLA, haciendo énfasis en la capacidad de adaptación, seguridad y coordinación entre vehículos autónomos bajo condiciones de lluvia.

Objetivos específicos

- Modelado del Entorno en CARLA: Configurar el simulador en búsqueda de un entorno con vehículos y personas NCPs, así como un estado lluvioso con depósitos de agua para simular escenarios desfavorables controlados.
- Formalización Matemática Dec-POMDP: Formular el problema de toma de decisiones como un Proceso de Decisión Markoviano Parcialmente Observable, definiendo espacios de estado globales, observaciones y un espacio de acciones continuas.
- Implementar una Arquitectura MAPPO(CTDE): Implementar el algoritmo Multi-Agent Proximal Policy Optimization con un esquema de Entrenamiento Centralizado con Ejecución Descentralizada.
- Diseño de Funciones de Recompensa: Diseñar y optimizar una función de recompensa adaptada que incentive la llegada a la meta, la posición centrada en el carril, una velocidad adecuada y penalice las colisiones y los desplazamientos laterales, así como la excesiva cercanía a vehículos y peatones.
- Estudio de ablación perceptual y de su convergencia: Realizar un análisis cualitativo estudiando el rendimiento del sistema y evaluando el espacio de observaciones con distintas conceptualizaciones, como pueden ser modelos visuales (cámara RGB y segmentación semántica) y sensores activos (Radar), así como analizando cuantitativamente mediante un vector de estado idealizado propio del simulador (*Ground Truth*) con el fin de clarificar la sensibilidad de la política al ruido perceptual y las limitaciones de convergencia de MAPPO en escenarios complejos.

Capítulo 1

Marco teórico

1.1. Fundamentos del Reinforcement Learning

El aprendizaje por refuerzo (*Reinforcement Learning*, RL) es un subcampo del machine learning en el que un agente trata de optimizar un proceso de toma de decisiones a través de la interacción con el entorno. Recibiendo recompensas por acciones realizadas correctamente, el agente aprenderá a maximizar esta recompensa. El objetivo final consiste en aprender una política que maximice la recompensa acumulada a largo plazo [7]. En la figura 1.1 se muestra visualmente este proceso de forma simplificada.

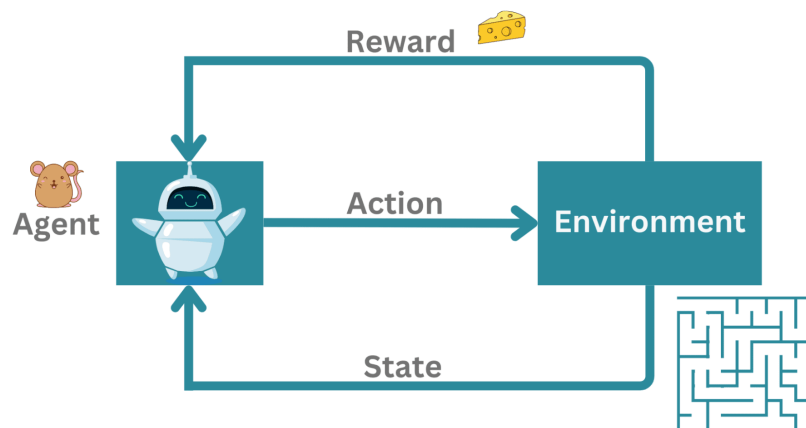


Figura 1.1: Proceso interactivo de un agente de Reinforcement Learning [8].

Este problema de toma de decisiones suele modelarse mediante un *Proceso de Decisión de Markov* (*Markov Decision Process*, MDP), que describe la interacción

entre el agente y el entorno a lo largo del tiempo. Un MDP puede definirse mediante la tupla [7]:

$$\langle S, A, P, R, \gamma \rangle$$

donde S representa el conjunto de estados del entorno, A el conjunto de acciones disponibles para el agente, $P(s'|s, a)$ la probabilidad de transición al estado s' tras ejecutar la acción a en el estado s , $R(s, a)$ la recompensa inmediata obtenida tras dicha transición y $\gamma \in [0, 1]$ el factor de descuento, encargado de ponderar la importancia de las recompensas futuras frente a las inmediatas.

Durante el proceso de aprendizaje, el agente selecciona acciones de acuerdo con una política π , la cual define la probabilidad de escoger una acción específica en cada estado [7]. El objetivo consiste en encontrar una política óptima que maximice la recompensa acumulada esperada.

La calidad de una política se obtiene mediante las funciones de valor, que estiman el retorno esperado asociado a un estado o a una acción [7]. La función acción-valor, $Q^\pi(s, a)$, representa la recompensa acumulada esperada al ejecutar la acción a en el estado s y continuar ejecutando acciones siguiendo la política π :

$$Q^\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

donde G_t representa el retorno acumulado descontado obtenido desde el instante t .

De forma análoga, la función estado-valor $V^\pi(s)$ expresa la recompensa acumulada esperada al comenzar en un estado s y seguir la política π [7]:

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a)$$

Ambas funciones constituyen la base del aprendizaje por refuerzo, ya que permiten estimar la calidad de las decisiones tomadas por el agente y de la política aprendida.

Las funciones anteriormente descritas se fundamentan en la ecuación de expectativa de Bellman [7], la cual establece una relación recursiva entre la recompensa

inmediata y los valores de los estados futuros. Matemáticamente, la función estado-valor se expresa como:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

1.2. Paradigmas de aprendizaje

A pesar de contar con las ecuaciones de Bellman, en la mayoría de los problemas reales el agente desconoce la dinámica del entorno (las probabilidades de transición $P(s'|s, a)$ y la función de recompensa R). Por tanto, debe aproximar las funciones de valor $V(s)$ y $Q(s, a)$ a través de prueba y error. La forma de recopilar experiencias, procesar las recompensas y optimizar el comportamiento por parte de un agente da lugar a diferentes estrategias y paradigmas de aprendizaje [7]:

- **Exploración y Explotación:** La exploración es un concepto fundamental en el aprendizaje por refuerzo y se refiere al comportamiento del agente para tratar de conseguir nueva información. Esto es útil porque el agente puede no tener apenas conocimiento de su entorno, por lo que probar diferentes estrategias puede ayudarle a comprender y tomar mejores decisiones. Por el contrario, la explotación consiste en, tras elegir una estrategia supuestamente óptima, usar ese conocimiento para tomar decisiones que retornen la mayor recompensa. La sobreexplotación puede llevar al agente a quedarse atrapado en un mínimo local, y la sobreexploración puede ser ineficiente, por lo que es crucial encontrar un buen balance entre ambas.
- **Monte Carlo y Diferencia Temporal:** Son dos métodos para estimar la función de valor. Monte Carlo estima esta función basándose en el aprendizaje post-episodio, actualizando el valor estimado a partir de la secuencia de recompensas obtenidas. Por su parte, los métodos basados en Diferencia Temporal actualizan el valor estimado a cada paso, lo que puede llevar a una convergencia más rápida y una menor varianza, aunque puede introducir sesgos.
- **On-Policy y Off-Policy:** Mientras que en los paradigmas *On-Policy* la política objetivo (aquella que el agente optimiza) y la política de comportamiento (la usada para la exploración) es la misma, en los métodos *Off-Policy* el agente usa una política para interactuar con el entorno y tomar acciones, mientras optimiza una política distinta aprendiendo de lo ocurrido.

1.3. Deep Reinforcement Learning

Los algoritmos y paradigmas descritos anteriormente, tanto tabulares que asumen implícitamente que las funciones de valor (V y Q) pueden almacenarse en tablas, como aquellos que emplean aproximaciones de funciones parametrizadas, presentan limitaciones para abordar problemas de gran complejidad. Esto es, cuando el espacio de estados o de acciones es de alta dimensión o continuo, estas representaciones resultan inviables. El Aprendizaje Por Refuerzo Profundo (*Deep Reinforcement Learning*, DRL) surge como una extensión al RL tradicional sustituyendo las tablas por redes neuronales profundas capaces de actuar como aproximadores universales de funciones [7, 9]. Gracias a esta aproximación, es posible abordar problemas complejos y desarrollar sistemas de aprendizaje de extremo a extremo (*end-to-end*), donde el agente aprende directamente a partir de la información sensorial proporcionada por el entorno.

El entrenamiento de estas redes neuronales requiere resolver un problema de optimización mediante el algoritmo de descenso por gradiente (*Gradient Descent*) [10]. Este algoritmo actualiza iterativamente los pesos de la red θ en la dirección opuesta al gradiente de la función de pérdida $f(\theta)$:

$$\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$$

donde α representa la tasa de aprendizaje (*learning rate*), que determina la magnitud del ajuste en cada iteración.

1.4. Principales Algoritmos

Los algoritmos de DRL pueden clasificarse en tres grandes familias según la forma en que representan y optimizan el comportamiento del agente: algoritmos basados en funciones de valor (*Value-Based*), algoritmos basados en políticas (*Policy-Based*) y algoritmos híbridos de tipo *Actor-Critic* [7]. La Figura 1.2 representa la relación entre la función de valor óptima $V_{\pi^*}(s)$ y la política óptima $\pi^*(s)$ en un entorno discreto. Cada celda de la izquierda de la imagen representa un estado y su valor esperado bajo la política óptima. Los valores aumentan al aproximarse al estado objetivo, cuyo valor es 0. A partir de estos valores se determina la acción óptima en cada estado, representada mediante las flechas en el panel de la derecha, que indican la dirección que permite avanzar hacia el objetivo de forma óptima.

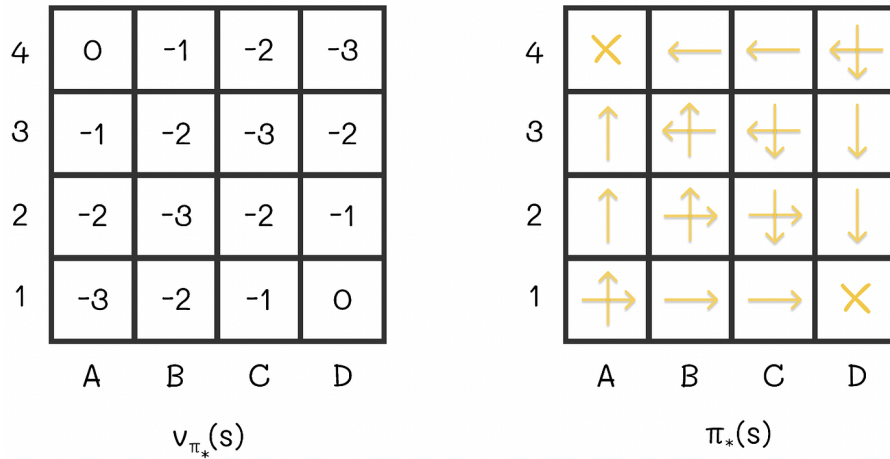


Figura 1.2: Diferencia entre algoritmos value-based y policy-based [11].

1.4.1. Algoritmos basados en funciones de valor (Value-Based)

Son algoritmos que se basan en aprender una aproximación de la función acción-valor $Q(s,a)$, estimando el retorno esperado al ejecutar una acción determinada en un estado concreto. La política se obtiene indirectamente al seleccionar la acción cuyo valor Q sea máximo [7].

Un ejemplo de ellos es Q-learning, un algoritmo off-policy que actualiza iterativamente la función acción-valor utilizando la recompensa obtenida y la mejor estimación del siguiente estado siguiendo:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

donde α representa la tasa de aprendizaje (*learning rate*) y γ el factor de descuento.

Para generalizar este método a espacios mucho mayores surge Deep Q-Network, sustituyendo la representación tabular por una red neuronal profunda que aproximará la función $Q(s,a)$ [9].

1.4.2. Algoritmos basados en políticas (Policy-Based)

Son algoritmos que, a diferencia de los anteriores, se basan en optimizar directamente la política $\pi(a|s)$ en lugar de aprender una función acción-valor. Esto permite trabajar de forma natural con espacios continuos.

Podemos mencionar Policy Gradient que optimiza directamente los parámetros de la política cuyo objetivo consiste en maximizar directamente el retorno esperado. La función objetivo puede expresarse como [12]:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[G]$$

donde θ representa los parámetros de la política. Tratará por tanto, de actualizar los parámetros mediante ascenso por gradiente (*Gradient Ascent*), actualizando los parámetros de la política tratando de favorecer las acciones que producen mayor retorno esperado y una mayor recompensa acumulada. Esta actualización de parámetros se puede expresar matemáticamente como [12]:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta_t),$$

donde α representa la tasa de aprendizaje.

1.4.3. Algoritmos Actor-Critic

Estos algoritmos se caracterizan por combinar las ventajas de los métodos basados en política y en valor. El Actor aprenderá la política responsable de tomar las acciones y el Critic se encargará de estimar una función de valor que sirva para evaluar estas decisiones y proporcionar feedback al Actor [13]. Cabe destacar algoritmos como A2C, TRPO, COMA y PPO [14]. Debido a su estabilidad, simplicidad de implementación y rendimiento en problemas continuos, este trabajo emplea PPO como algoritmo. En la siguiente sección se describen sus principales características.

1.5. Proximal Policy Optimization (PPO)

Como se ha presentado, los algoritmos basados en gradientes de política actualizan directamente los parámetros para maximizar el retorno esperado. Sin embargo, si las actualizaciones son demasiado grandes, producirán una pérdida significativa de rendimiento. Surge así *Trust Region Policy Optimization* (TRPO) [15], limitando el tamaño de las actualizaciones pero con la dificultad de requerir la resolución de un problema de optimización restringida en cada iteración. Como alternativa aparece *Proximal Policy Optimization* (PPO) [16], proponiendo una aproximación mucho mas sencilla. Mantiene la estabilidad modificando la función objetivo. En lugar de imponer una restricción explícita como TRPO, PPO introduce un *clipping* que limita

el tamaño de las actualizaciones sobre la política. Es decir, así como *Policy Gradient*, actualiza los parámetros de la política mediante ascenso por gradiente siguiendo [16]:

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta} J(\theta_t),$$

donde α representa la tasa de aprendizaje.

PPO modifica esta actualización mediante el cociente de la probabilidad de seleccionar una acción bajo la nueva política y bajo la política anterior [16]:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}.$$

Además, utiliza la función de ventaja (*Advantage Function*), que mide cuánto mejor o peor resulta una acción respecto al valor esperado del estado [16]:

$$A_t = Q(s_t, a_t) - V(s_t).$$

A partir de estos términos, PPO define la siguiente función objetivo [16]:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} [\text{mín}(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)].$$

El operador *clip* restringe el valor de $r_t(\theta)$ al intervalo $[1 - \epsilon, 1 + \epsilon]$, evitando que la nueva política difiera excesivamente de la anterior. De esta forma, si una actualización intenta modificar demasiado la probabilidad de una acción, PPO la limita automáticamente.

Por su estabilidad, simplicidad de implementación y rendimiento, PPO se ha convertido, frente a otros expuestos, en uno de los algoritmos más utilizados en problemas de control continuo y constituye la base del algoritmo MAPPO empleado en este trabajo.

1.6. Multi-Agent Deep Reinforcement Learning

El *Multi-Agent Reinforcement Learning* (MARL) extiende el aprendizaje por refuerzo tradicional a escenarios en los que múltiples agentes interactúan. A diferencia del caso de un único agente, cada acción tomada por un agente puede implicar cambios en el resto, convirtiendo el problema en un entorno dinámico y no estacionario para cada agente [17].

Esto es la base para tratar problemas de coordinación y cooperación presentes en numerosos escenarios, como la conducción autónoma, la navegación de enjambres de robots o la gestión del tráfico [14]. En el ámbito de la conducción autónoma, MARL ha demostrado ser especialmente útil para resolver maniobras cooperativas como incorporaciones, cruces e intersecciones, siendo esencial para mejorar tanto la seguridad como la fluidez del tráfico. Sin embargo, el aprendizaje multiagente introduce nuevos desafíos como es la no estacionariedad del entorno, ya que, como se ha comentado, cada agente modifica continuamente su política durante el entrenamiento y como consecuencia, el entorno observado por un agente deja de ser estático, dificultando la convergencia de los algoritmos. Además, el incremento del número de agentes provoca un crecimiento exponencial del espacio de estados y de las posibles interacciones entre los agentes.

Con el objetivo de afrontar estos problemas, se definen diferentes paradigmas, entre los que destacan los siguientes:

- IPPO (Independent PPO): Cada agente ignora la existencia de los demás agentes y entrena su propio modelo. Aunque la implementación es más sencilla que el resto, sufre de inestabilidad por la no-estacionariedad, ya que considera a los agentes parte del entorno [18].
- CTDE (Centralized Training with Decentralized Execution): En este primer paradigma, durante el entrenamiento, los agentes comparten información con un crítico centralizado que luego se usará para actualizar la política de cada uno de ellos. De esta manera, conviertes un entorno parcialmente observable (cada agente solo ve una parte del entorno) al usar un crítico centralizado que dispone de información global del entorno y de todos los agentes, permitiendo estimar funciones de valor más precisas y estabilizar el aprendizaje. Durante la ejecución los agentes solo parten de sus observaciones locales y no necesitan comunicación adicional [19, 20].
- DTDE (Decentralized Training with Decentralized Execution): En este paradigma cada agente opera independientemente, sin información global. Esto puede robustecerse con comunicación entre agentes para equipararse a la completitud que despliega CTDE [18].

En la Figura 1.3 se puede apreciar la diferencia entre el paradigma CTDE y el DTDE. En CTDE se utiliza un controlador central para concatenar observaciones y equilibrar las políticas y en DTDE el agente actualiza la política independientemente él mismo.

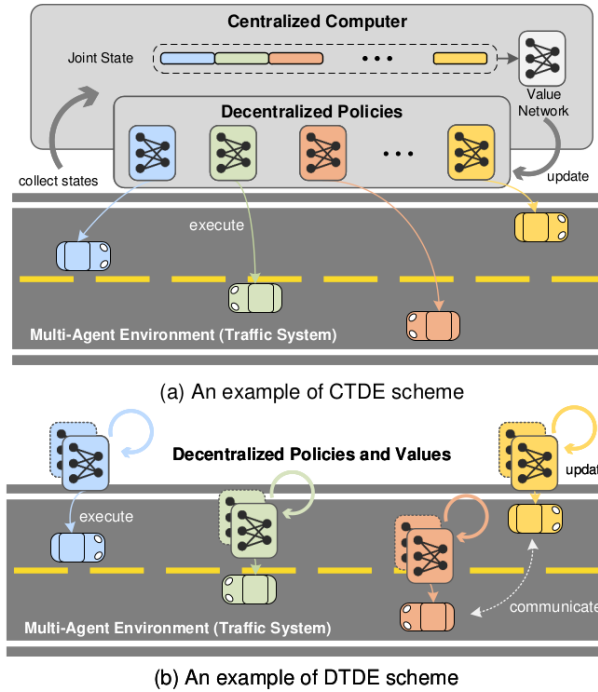


Figura 1.3: Diferencia de los paradigmas de entrenamiento y ejecución [14]. En la imagen (a) se aprecia la descentralización de las políticas de cada agente y un crítico que recibe observaciones de todos ellos para estimar la función de valor general. Por su parte, en (b) la parte del crítico desaparece, y con él la figura que maximiza las acciones del conjunto de actores

1.6.1. Multi-Agent Proximal Policy Optimization (MAPPO)

Entre los algoritmos desarrollados bajo el paradigma CTDE destaca Multi-Agent Proximal Policy Optimization (MAPPO) [18], una extensión del algoritmo PPO. MAPPO permite a cada agente ejecutar una política utilizando únicamente sus observaciones locales, mientras que un crítico centralizado emplea información global del entorno durante el entrenamiento para estimar funciones de valor más precisas.

Gracias a esta arquitectura, MAPPO consigue combinar la estabilidad de PPO con las ventajas del entrenamiento centralizado. Busca evitar la no-estacionariedad y conseguir comportamientos cooperativos entre los agentes. Estas características han convertido a MAPPO en uno de los algoritmos de referencia para problemas cooperativos con espacios de acciones continuos.

Una vez establecidos los fundamentos teóricos necesarios, el siguiente capítulo analiza el estado actual de la investigación en conducción autónoma basada en aprendizaje por refuerzo, identificando las principales limitaciones existentes y la motivación del presente trabajo.

Capítulo 2

Estado del arte

2.1. Aprendizaje por refuerzo aplicado a la conducción autónoma

Durante el reciente crecimiento de Reinforcement Learning respecto a la conducción autónoma se han llevado a cabo diferentes investigaciones que han derivado en un avance notorio del mismo.

Se ha demostrado que un vehículo puede aprender una política para mantenerse en el carril usando DDPG en solo 15 o 20 minutos en el mundo real. En el trabajo de Kendall et al. [21] la entrada era solamente una cámara frontal y un sensor de velocidad. Durante la prueba llevada a cabo, el coche aprendió a mantenerse en carreteras con curvas suaves y condiciones de tráfico ligeras dentro del carril y además mostró capacidad de generalización a nuevas carreteras. Se usó una función de recompensa que incentivaba mantener una velocidad constante, mantenerse dentro del carril, avanzar metros y no colisionar. Consiguieron mejores resultados al usar DDPG + VAE(variational autoencoders), es decir primero preentrenando una red reduciendo una imagen a un vector pequeño y denso.

Posteriormente, otros trabajos comenzaron a estudiar la influencia de las condiciones meteorológicas. En este sentido, Ozturk et al. [22], exploraron como varía la conducción en lluvia, nieve y seco en un coche con un modelo de RL. Resaltando la degradación del algoritmo en lluvia, donde la pérdida de información, como puede ocurrir con la visión por computador, es el factor más determinante. Se reducía así la recompensa obtenida por el agente en lluvia con respecto a condiciones secas hasta un 54 % en rectas, un 28 % en giros en U y un 22 % en circuitos de carreras. Resulta

interesante la extensión de esto a multiagente buscando solucionar este problema, ya que puede que la visión parcial de cada agente del entorno ayude a entender a los demás agentes el mismo.

2.2. Aprendizaje por refuerzo multiagente en conducción autónoma

Aunque los algoritmos de aprendizaje por refuerzo han demostrado ser capaces de resolver diversas y complejas tareas, la mayoría de estos trabajos se centraban en un único vehículo interactuando con el entorno. Sin embargo, el tráfico real está compuesto por múltiples vehículos que toman decisiones interdependientes, por lo que resulta necesario extender estos métodos hacia el aprendizaje por refuerzo multiagente (*Multi-Agent Reinforcement Learning*, MARL).

Surge un avance fundamental en esta área al introducir el paradigma CTDE como se muestra en Gupta et al. [23]. Se apreciaron grandes beneficios en tener un crítico centralizado con actores descentralizados, demostrando así que es posible lograr la coordinación multiagente mientras se mantiene la ejecución independiente.

Otros estudios presentan la efectividad de diversos algoritmos MARL en varias situaciones conflictivas como pueden ser la incorporación a una carretera o el adelantamiento resultando en que, para una carretera bidireccional, MADDPG es el algoritmo con mejores resultados, así como en intersecciones; mientras que para la fusión de carriles PPO se comporta mejor debido a que provee mayor estabilidad en entornos dinámicos [24].

La mayor parte de los trabajos sobre aprendizaje por refuerzo multiagente para conducción autónoma se desarrollan en condiciones ambientales favorables, con el objetivo de reducir la complejidad perceptual del problema y facilitar el proceso de aprendizaje. Como consecuencia, la evaluación de los algoritmos suele realizarse en escenarios alejados de las condiciones que pueden encontrarse en la conducción real.

Esta limitación queda reflejada en la revisión presentada por Zhang et al. [14], donde se analizan numerosos escenarios de conducción cooperativa y se comparan diferentes algoritmos MARL. El estudio identifica a MADDPG y MAPPO como dos de los enfoques con mejores resultados en múltiples tareas de conducción autónoma. Sin embargo, también pone de manifiesto la escasez de trabajos que evalúen estos algoritmos bajo condiciones meteorológicas adversas o en escenarios perceptualmente

te más complejos, lo que evidencia una importante línea de investigación todavía abierta.

Paralelamente, Schmidt et al. [25] recoge algunos de los principales algoritmos empleados actualmente en conducción autónoma cooperativa, entre los que destacan COMA, basado en mecanismos de asignación de crédito, NeurComm, que incorpora comunicación vehículo a vehículo (V2V), y los enfoques basados en entrenamiento centralizado con ejecución descentralizada, como MADDPG y MAPPO.

Por lo general, los estudios nos muestran que usar MAPPO para la planificación de alto nivel y MPC+CBF para control seguro de bajo nivel es lo más oportuno, demostrando que la combinación de esto logra un balance entre seguridad y eficiencia muy interesante [26].

A partir del análisis de estos trabajos, en el presente proyecto se estudiaron diferentes alternativas, entre ellas SAC, COMA, MAAC, MADDPG y MAPPO. Finalmente, se seleccionó MAPPO como algoritmo de aprendizaje debido a la estabilidad heredada de PPO, su compatibilidad con el paradigma CTDE y el elevado rendimiento mostrado en problemas cooperativos de conducción autónoma. Por el lado de comunicación entre agentes, aunque los resultados en las condiciones climatológicas más favorables parecen ser mejores en caso de incluir un número N limitado de agentes. En condiciones meteorológicas adversas, los canales de comunicación V2V pueden verse comprometidos por interferencias, latencia variable o fallos completos. Además, la escalabilidad para un número de agentes $A > N$ dificulta la gestión de esta comunicación y la información útil a interpretar [2].

2.3. Problemas en Condiciones Meteorológicas Adversas

Uno de los problemas más críticos en la conducción autónoma, como se ha ido arrastrando en el presente estudio, viene dado por la dificultad adicional que suponen las condiciones meteorológicas adversas, debido principalmente a la degradación en el rendimiento de los sensores.

- Lidar: Este sensor (que primero transmite un pulso láser y luego mide el tiempo hasta que recibe su reflexión) sufre dispersión debido a las partículas de agua, lo que conduce a falsas detecciones [27].

- **Cámara:** Por su parte, las cámaras sufren fluctuaciones bruscas en la intensidad de la luz, lo que lleva a una degradación significativa de la calidad de las imágenes. Aunque estos efectos pueden reducirse mediante preprocesamiento o ajustando parámetros de la cámara, el ruido producido por la lluvia sigue influyendo en gran medida [28].
- **Radar:** El radar también se ve afectado por efectos de dispersión y atenuación, ya que la lluvia causa interferencia con las ondas electromagnéticas que se emiten y reciben (midiendo el tiempo de retorno) [29].

2.4. Análisis del estado del arte actual

A partir del análisis realizado, se identifican tres limitaciones principales en el estado del arte actual:

- **Predominio de escenarios con un solo agente:** La mayoría de los trabajos se centran en el control de un único vehículo, obviando problemas de no estacionariedad derivados de la interacción entre agentes.
- **Escasez de estudios de ablación perceptual:** Existen pocos análisis en los que se evalúe cómo influye el tipo de sensor (ruido visual vs. datos idealizados) en la capacidad de aprendizaje de las políticas de RL.
- **Evaluación limitada con físicas dinámicas adversas:** Pocos entornos de prueba combinan simulación de física con baja adherencia (asfalto mojado) y tráfico dinámico.

Aunque existen trabajos que estudian de forma independiente el aprendizaje por refuerzo, el aprendizaje multiagente o el impacto de las condiciones meteorológicas sobre la conducción autónoma, son muy escasas las investigaciones que combinan simultáneamente estos tres aspectos utilizando simuladores de alta fidelidad.

El presente Trabajo Fin de Máster pretende contribuir a reducir esta brecha mediante el desarrollo de un sistema basado en MAPPO bajo el paradigma CTDE sobre el simulador CARLA. El estudio analiza la ablación perceptual al comparar representaciones basadas en visión profunda (RGB y segmentación semántica), sensores activos (Radar) y métricas idealizadas de referencia (*Ground Truth*). Buscando así aislar las fuentes de incertidumbre perceptual y evaluar las limitaciones reales de convergencia de la política en maniobras de evasión y seguimiento en condiciones extremas de lluvia.

Capítulo 3

Gestión de proyecto software

3.1. Alcance del proyecto

3.1.1. Definición del proyecto

El trabajo de Fin de Máster que se abarca tiene como objetivo el diseño, implementación y evaluación de un sistema de aprendizaje por refuerzo profundo multi-agente (*Multi-Agent Deep Reinforcement Learning*, MARL) orientado a la conducción autónoma en un entorno simulado. Se desarrolla sobre el simulador CARLA y emplea el algoritmo *Multi-Agent Proximal Policy Optimization* (MAPPO) que sigue el paradigma *Centralized Training with Decentralized Execution* (CTDE) expuesto anteriormente.

El alcance de este proyecto comprende el diseño de un entorno de conducción en el que varios vehículos autónomos pueden aprender de forma cooperativa mediante interacción con el entorno. Es por ello que se define el problema como un proceso de decisión de Markov parcialmente observable multiagente (*Decentralized Partially Observable Markov Decision Process*, Dec-POMDP). También engloba la implementación de los espacios de observación y acción y la función de recompensa empleada durante el aprendizaje.

La implementación por tanto incluye la integración del simulador CARLA con el entorno, la generación del espacio de observaciones de los agentes a partir de información del vehículo y del Ground Truth o, durante la evaluación, el sensor radar del simulador para obtener distancias a obstáculos, la definición de las políticas de cada agente y del crítico centralizado y el desarrollo del proceso de entrenamiento mediante MAPPO. Asimismo, se contempla la utilización de condiciones ambien-

tales desfavorables, siendo estas lluvia y depósitos de agua en la calzada, con el objetivo de estudiar el comportamiento del sistema bajo condiciones de percepción más complejas.

Inicialmente, el sistema se plantea para dos vehículos autónomos, buscando comprobar el funcionamiento del enfoque multiagente y siendo extensible el algoritmo MAPPO a incrementar el número de agentes mediante la sustitución de las distancias entre ellos por Graph Neural Networks si la capacidad computacional y la estabilidad del entrenamiento lo permiten. El entorno también puede incorporar vehículos y peatones controlados por el simulador para aumentar la complejidad del escenario y aproximarlos a una situación de tráfico dinámica real.

El proyecto comprende finalmente la realización de experimentos con el fin de evaluar el aprendizaje de los agentes y su comportamiento durante la conducción. Se analiza así la capacidad de los vehículos para mantener la trayectoria, alcanzar los destinos establecidos, mantener una velocidad adecuada y evitar colisiones, además de la influencia de las condiciones meteorológicas sobre el rendimiento del sistema.

Queda fuera del alcance del proyecto la implementación de un sistema de conducción autónoma sobre vehículos físicos y la utilización de comunicación V2V real entre vehículos y el desarrollo de sensores físicos. Para concluir, el objetivo no es desarrollar un sistema de conducción autónoma completo para entornos reales, sino estudiar la aplicación de MARL, en específico MAPPO, en un entorno simulado controlado.

3.2. Metodología de trabajo

Para la organización y seguimiento del desarrollo del proyecto se ha empleado una metodología de trabajo basada en principios de metodologías ágiles, tomando como referencia Scrum. Esta elección se debe a la naturaleza experimental del proyecto, lo que deriva en que las decisiones de implementación y las siguientes líneas de trabajo dependen de los resultados obtenidos durante el desarrollo.

El trabajo se ha organizado en iteraciones de aproximadamente dos semanas, fijando reuniones periódicas con el tutor de dicho trabajo. En estas reuniones se revisaba el trabajo realizado durante el periodo anterior, se analizaban los problemas encontrados y se establecían los objetivos y tareas prioritarias para la siguiente iteración.

Este planteamiento permitió mantener un desarrollo incremental y adaptar la planificación en función de los resultados obtenidos, algo especialmente relevante en un proyecto de investigación basado en aprendizaje por refuerzo, donde no siempre es posible conocer de antemano la duración o el resultado de las distintas fases de experimentación.

3.2.1. Identificación de tareas

Para la planificación se dividió el desarrollo del Trabajo Fin de Máster en diferentes fases, agrupando cada actividad realizada en función de su objetivo y dependencia. Esta división facilita estimar los recursos necesarios y establecer una planificación del trabajo.

Las principales actividades identificadas y los recursos para llevarlas a cabo son las que se pueden identificar en la Tabla 3.1:

Tabla 3.1: Estimación de tareas.

Fase	Tareas principales	Recursos
Revisión y estudio inicial	Estudio de Reinforcement Learning, Deep Reinforcement Learning y Multi-Agent Reinforcement Learning. Revisión de algoritmos como PPO, MAP-PO y otros enfoques MARL. Análisis del estado del arte en conducción autónoma.	Bibliografía
Configuración del entorno	Configuración de CARLA, integración con Python, configuración de mapas, vehículos, sensores, tráfico y condiciones meteorológicas.	CARLA, Python
Diseño del problema	Definición del escenario, formulación como Dec-POMDP, definición del espacio de observaciones, espacio de acciones y función de recompensa.	Python, CARLA

Continúa en la siguiente página...

Continuación de la página anterior

Fase	Tareas principales	Recursos
Implementación	Desarrollo del entorno de aprendizaje, implementación de los agentes, redes Actor-Critic, algoritmo MAPPO, cálculo de ventajas y proceso de actualización de las redes.	Python, PyTorch, CARLA, GPU
Experimentación	Entrenamiento de los agentes, ajuste de hiperparámetros, análisis de la estabilidad del aprendizaje y realización de experimentos bajo diferentes condiciones.	Servidor, GPU NVIDIA L4, CARLA
Evaluación	Análisis del comportamiento de los agentes, estudio de las recompensas obtenidas, capacidad de navegación, mantenimiento del carril y aparición de colisiones.	Python, PyTorch, herramientas de análisis
Documentación	Redacción del estado del arte, marco teórico, metodología, resultados, conclusiones y documentación del desarrollo realizado.	L ^A T _E X

3.2.2. Estimación de tareas

La Tabla 3.2 recoge una estimación de la dedicación necesaria para cada una de estas actividades.

Tabla 3.2: Planificación y estimación de tareas por Sprints.

Sprint / Tarea	Semanas	Dedicación
Sprint 1: Fundamentos y Entorno		
Estudio del estado del arte y fundamentos	S1 – S2	25 h
Configuración de CARLA y entorno	S2 – S3	25 h
Sprint 2: Diseño del Escenario		
Diseño de escenario, observaciones y recompensas	S4 – S6	40 h
Sprint 3: Desarrollo Algorítmico		

Continúa en la siguiente página...

Continuación de la página anterior

Sprint / Tarea	Semanas	Dedicación
Implementación de MAPPO, inclusión de distintos tipos de sensores y de observaciones idealizadas	S7 – S10	45 h
Sprint 4: Experimentación		
Entrenamiento, ajustes y evaluación	S10 – S14	60 h
Tareas Transversales		
Redacción y revisión de la memoria	S5 – S15	30 h
Total estimado		225 h

3.3. Plan de trabajo

La ejecución del proyecto se ha organizado a lo largo de un periodo de 15 semanas, divididas en cuatro Sprints consecutivos de desarrollo técnico y un bloque de tareas transversales de documentación. Esta metodología ágil permite abordar el problema de forma iterativa, validando los componentes del sistema a medida que se incrementa la complejidad del entorno y del modelo.

En la Figura 3.1 se detalla la planificación temporal, especificando la duración y solapamiento de las tareas principales reflejadas en la estimación de dedicación.

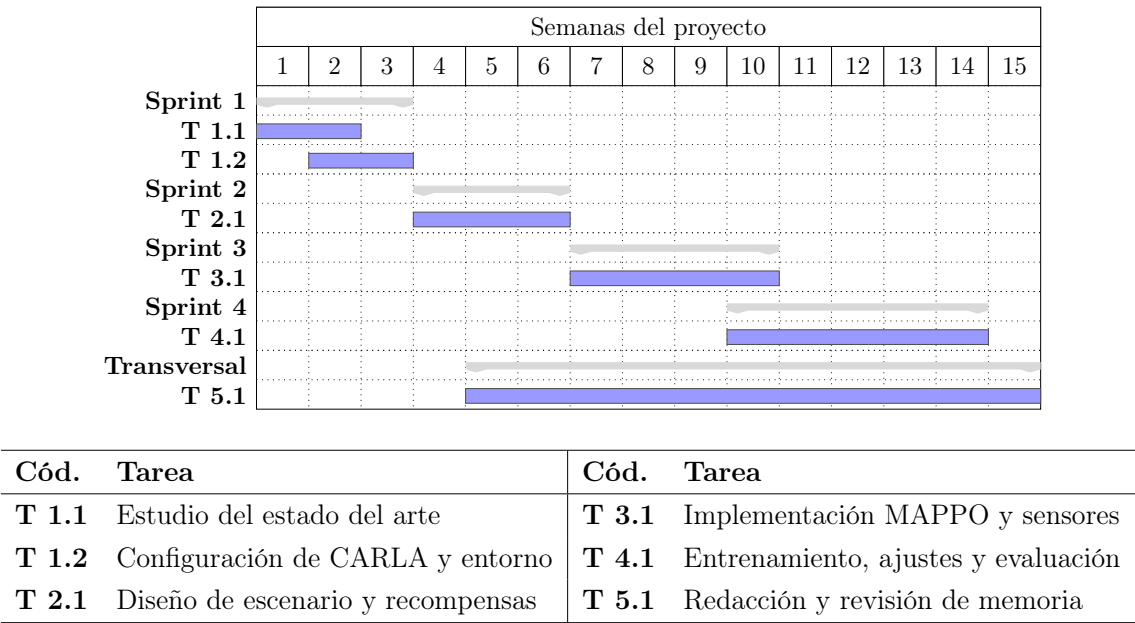


Figura 3.1: Diagrama de Gantt.

3.4. Gestión de recursos

Para el desarrollo del proyecto se requieren recursos humanos, hardware y software. Al tratarse de un proyecto de investigación, el principal recurso corresponde al tiempo de trabajo dedicado al diseño, implementación, experimentación y análisis de los resultados.

3.4.1. Presupuesto

El presupuesto del proyecto es una estimación que engloba todos los costes asociados a los recursos necesarios para alcanzar los objetivos definidos. Se realiza una valoración económica para simular un entorno profesional y poner valor del trabajo realizado. Los costes se dividen en las siguientes categorías:

Coste de personal

En este proyecto el principal recurso humano necesario es un ingeniero especializado en inteligencia artificial y robótica, responsable del diseño, implementación y evaluación del sistema MAPPO sobre el simulador CARLA y un Scrum Master para la gestión ágil del desarrollo.

Para realizar la estimación, se ha tenido en cuenta que el salario medio para un ingeniero de Machine Learning en España varía entre 40.700 y 51.700 € [30] luego se ha impuesto un salario, suponiendo 3 años de experiencia, de 43.000,00€ anuales. Además, la jornada laboral se supone de 40 horas semanales, luego se considera un coste de 22,37 €/hora. El número de horas se obtiene a partir de la estimación de tareas realizada en el apartado anterior. Por su lado, el salario medio para el puesto de Scrum Master es aproximadamente de 45.000,00€ [31], es decir en 40 horas laborales, implicando un coste de 23,41 €/h. Para la labor de seguimiento y sincronización, se estima una dedicación de 1 hora cada 2 semanas a lo largo de las 15 semanas de duración del proyecto, sumando un total de 7,5 horas de dedicación.

En la Tabla 3.3 se muestra el coste estimado del personal.

Tabla 3.3: Presupuesto estimado para los recursos humanos.

Recurso	Horas	Coste/hora	Total
Ingeniero IA/Robótica	225,0	22,37 €	5.033,25 €
Scrum Master	7,5	23,41 €	175,58 €
Total			5.208,83 €

Coste de infraestructura computacional

El entrenamiento de los agentes requiere una infraestructura capaz de ejecutar simultáneamente el simulador CARLA y las redes neuronales empleadas durante el aprendizaje. Se considera como recurso directo el uso de un servidor equipado con una GPU dedicada.

En el desarrollo del proyecto se ha utilizado la infraestructura computacional proporcionada por la universidad. Por tanto, no se considera el servidor como una inversión directa, sino el coste estimado correspondiente a su utilización durante el proyecto.

Para simplificar la estimación, se establece un coste de utilización de 2,00 €/h. Considerando una utilización estimada de 150 horas para las tareas de entrenamiento y experimentación, el coste asociado a la infraestructura computacional es:

$$C_{\text{GPU}} = 2,00 \cdot 150 = 300,00$$

Así, se puede apreciar en la Tabla 3.4 el costo de esta infraestructura

Tabla 3.4: Coste estimado de la infraestructura computacional.

Recurso	Coste/hora	Horas	Total
Servidor con GPU	2,00 €	150	300,00 €
Total			300,00 €

Coste de software y licencias

Una de las ventajas del proyecto es que se apoya principalmente en herramientas y tecnologías de código abierto. El simulador CARLA, Python, PyTorch, Gym/Gymnasium y las principales bibliotecas empleadas para el desarrollo no requieren licencias comerciales para el uso considerado.

Por tanto, el coste directo asociado al software es únicamente el asociado a herramientas que optimicen el desarrollo. En este caso, se usó GitHub Copilot, una herramienta de Microsoft [32], siendo su coste el recogido en la Tabla 3.5

Tabla 3.5: Presupuesto estimado para software y licencias.

Concepto	Coste mensual	Meses	Total
GitHub Copilot	10,00 €	4	40,00 €

Costes indirectos

Los costes indirectos corresponden a los recursos imprevistos o amortizaciones de equipamiento informático secundario necesarios para la ejecución del proyecto que no pueden asociarse a una tarea concreta. Se calcula como un 5 % [33].

$$C_{\text{indirectos}} = 0.05 \cdot C_{\text{directos}}$$

Costes generales

Los costes generales representan cualquier gasto asociados al funcionamiento de la infraestructura necesaria para desarrollar el proyecto, incluyendo internet, electricidad, mantenimiento de instalaciones y otros gastos generales. Se establece un coste equivalente al 15 % del Presupuesto de Ejecución Material (PEM) [33].

$$C_{\text{generales}} = 0.15 \cdot PEM$$

Beneficio industrial

El beneficio industrial representa el margen económico que una empresa aplicaría sobre los costes asociados al desarrollo del proyecto. Para esta estimación se considera un margen del 12 % sobre el Presupuesto de Ejecución Material [33].

$$C_{\text{beneficio}} = 0.12 \cdot PEM$$

IVA

Finalmente, se considera el Impuesto sobre el Valor Añadido (IVA) correspondiente a la prestación de servicios. Conforme a la legislación vigente, se aplica el tipo general, situado en el 21 % [34] sobre la base imponible resultante de los costes anteriores.

$$IVA = 0.21 \cdot (C_{\text{directos}} + C_{\text{indirectos}} + C_{\text{generales}} + C_{\text{beneficio}})$$

Coste total

Con todo esto, se puede resumir el presupuesto total estimado de la manera que se muestra en la Tabla 3.6:

Tabla 3.6: Presupuesto final estimado del proyecto.

Concepto	Porcentaje	Coste
Costes de personal		5.208,83 €
Costes de software		40,00 €
Costes de GPU		300,00 €
Costes directos		5.548,83 €
Costes indirectos	5 %	277,44 €
Presupuesto de Ejecución Material (PEM)		5.826,27 €
Costes generales	15 %	873,94 €
Beneficio industrial	12 %	699,15 €
Subtotal (Base Imponible)		7.399,36 €
IVA	21 %	1.553,87 €
Total		8.953,23 €

Ascendiendo por tanto a un total de 8.953,23 € incluyendo costes directos, indirectos, generales, beneficio industrial e IVA.

3.5. Gestión de riesgos

Durante el desarrollo del proyecto se han identificado diferentes riesgos asociados tanto a la implementación del sistema como al proceso de entrenamiento y evaluación. Estos riesgos adquieren especial relevancia debido al carácter experimental del aprendizaje por refuerzo y a la dependencia del simulador CARLA y de los recursos computacionales.

Para cada riesgo se ha estimado su probabilidad de ocurrencia y el impacto que podría tener sobre el desarrollo del proyecto. Asimismo, se han definido medidas de gestión orientadas a reducir su probabilidad o mitigar sus consecuencias.

3.5.1. Identificación de riesgos

La Tabla 3.7 recoge los principales riesgos identificados.

Tabla 3.7: Análisis de riesgos del proyecto.

Riesgo	Categoría	Prob.	Impacto	Causas	Gestión
Inestabilidad durante el entrenamiento	Implement	Alta	Alto	Variaciones en las recompensas o falta de convergencia durante la optimización de MAPPO.	Ajuste de hiperparámetros, normalización de ventajas, <i>gradient clipping</i> y monitorización del entrenamiento.
Convergencia insuficiente de MAPPO	Implement	Media	Alto	Complejidad del entorno, observaciones multimodales e interacción entre agentes.	Simplificación progresiva del entorno, reducción de agentes y ajuste de la función de recompensa.
Complejidad computacional elevada	Recursos	Alta	Alto	Renderizado de CARLA, procesamiento de imágenes convolucionales o análisis matricial de píxeles de segmentación y entrenamiento simultáneo de las redes neuronales.	Sustituir el procesamiento visual primero por un radar y finalmente por un vector escalar basado en la cinemática exacta, limitar NPCs, ajustar el tamaño de lote y aprovechar la GPU disponible.
versiones no compatibles de CARLA	Implement	Media	Medio	Diferencias entre versiones del simulador o entre cliente y servidor.	Fijar las versiones utilizadas y documentar la configuración del entorno.
Fallos o inestabilidad del simulador CARLA	Implementación	Media	Alto	Caídas del servidor, errores de sensores o problemas de comunicación con el simulador.	Reinicio del simulador, comprobación de sensores y guardado periódico de los modelos y métricas.
Diseño inadecuado de la función de recompensa	Diseño	Alta	Alto	Una ponderación incorrecta puede generar comportamientos no deseados durante el aprendizaje.	Diseñar la recompensa mediante componentes independientes y analizar su contribución durante el entrenamiento.
Continúa en la siguiente página...					

Riesgo	Categoría	Prob.	Impacto	Causas	Gestión
Degradación de la percepción bajo lluvia	Entorno	Alta	Alto	Las condiciones meteorológicas adversas pueden degradar la información proporcionada por los sensores.	Abstraer el bloque de percepción utilizando la información de referencia del simulador.
Complejidad al aumentar el número de agentes	Diseño	Alta	Medio	El espacio de estados y las interacciones aumentan con el número de vehículos.	Comenzar con dos agentes y aumentar progresivamente su número si la estabilidad lo permite.
Errores de integración entre CARLA y el entorno RL	Implement	Media	Alto	Problemas de comunicación, sincronización o adquisición de datos de los sensores.	Realizar pruebas independientes de cada componente y validar progresivamente la integración.
Insuficiencia de recursos GPU	Recursos	Media	Alto	La memoria y capacidad de procesamiento pueden resultar insuficientes para el entrenamiento.	Reducir tamaño de lote y resolución de imágenes y utilizar servidores con GPU cuando sea necesario.
Desviación respecto a los plazos previstos	Planing	Media	Medio	Los tiempos de entrenamiento y depuración pueden superar las estimaciones iniciales.	Establecer hitos intermedios, priorizar objetivos y reservar tiempo para experimentación y depuración.

3.6. Legislación y normativa

En el marco de ejecución de este proyecto, se ha llevado a cabo un riguroso cumplimiento de la legislación y normativa vigente. A continuación, se detalla cómo el proyecto se ajusta y adhiere a las leyes pertinentes:

- **Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales [35]**

Este proyecto respeta plenamente la Ley Orgánica 3/2018, la cual reconoce el derecho fundamental a la protección de datos personales. La creación del dataset se ha desarrollado de acuerdo con las disposiciones de la ley, asegurando

la legalidad en el tratamiento de datos. Los permisos explícitos obtenidos para el uso de imágenes garantizan la privacidad de los individuos. Además, las pruebas y el desarrollo se realizaron en un entorno controlado para evitar perjuicios en materia de datos a terceros. Aunque las imágenes capturadas por la cámara del robot no se almacenan, se garantiza que el procesamiento cumple con los principios de protección de datos establecidos en la ley.

- **Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (RGPD) [36]**

La creación del dataset en este proyecto se ajusta a los principios del RGPD, asegurando la legalidad y transparencia en el tratamiento de datos personales. Se han implementado medidas técnicas y organizativas para garantizar la seguridad y privacidad de los datos presentes en el dataset, cumpliendo así con las normativas del RGPD.

- **Propuesta de Reglamento del Parlamento Europeo y del Consejo sobre Inteligencia Artificial (PRLIA) [37]**

El proyecto ha sido diseñado y ejecutado considerando de manera integral las disposiciones y requisitos establecidos por la PRLIA. Un documento legislativo propuesto por la Unión Europea para regular el uso de la inteligencia artificial en diversos ámbitos. Esta propuesta tiene como objetivo establecer normas armonizadas en materia de inteligencia artificial para garantizar la seguridad, ética y transparencia en su desarrollo y aplicación. Aunque el proyecto no involucra una IA de alto riesgo, se han tenido en cuenta las recomendaciones éticas reflejadas en el marco ético del proyecto. No se ha incumplido ninguna disposición de la PRLIA, ya que no se han violado derechos fundamentales ni se han utilizado técnicas subliminales o vulnerabilidades de grupos específicos.

- **Real Decreto Legislativo 1/1996 sobre Propiedad Intelectual de 12 de abril, por el que se aprueba el texto refundido de la Ley de Propiedad Intelectual, regularizando, aclarando y armonizando las disposiciones legales vigentes sobre la materia. [38]**

En conformidad con el Real Decreto Legislativo 1/1996, el proyecto respeta la normativa sobre propiedad intelectual. Se ha optado por utilizar únicamente código y herramientas de software libre y de código abierto para garantizar el cumplimiento de la normativa en materia de propiedad intelectual.

Capítulo 4

Metodología

El código fuente completo con la implementación de los entornos, espacios de observaciones y acciones, función de recompensa y scripts de entrenamiento se encuentra disponible públicamente en GitHub [39].

4.1. Selección del entorno de simulación

La conducción autónoma es un problema que requiere integrar varios subsistemas para afrontar entornos dinámicos y parcialmente observables. Todo esto puede llevar a costes elevados y de baja seguridad por lo que la investigación en este campo se apoya en el uso de simuladores. En los últimos años han surgido diversos simuladores orientados al desarrollo y evaluación de estos sistemas. Entre todos destacan los que se aprecian en la Tabla 4.1 [14]:

Tabla 4.1: Comparativa de simuladores utilizados en aprendizaje por refuerzo para conducción autónoma.

Simulador	Tipo / Entorno	Características Visuales y Físicas	Enfoque en RL / MARL
TORCS	Carreras de coches	Gráficos 3D sencillos y físicas simplificadas	Pionero en RL mono-agente; ligero en cómputo pero limitado en tráfico urbano.

Continúa en la siguiente página...

Continuación de la página anterior

Simulador	Tipo / Entorno	Características Visuales y Físicas	Enfoque en RL / MARL
SUMO	Tráfico urbano microscópico	Simulación 2D basada en grafos sin renderizado 3D	Excelente para flujo de tráfico y MARL a gran escala; sin sensores visuales.
SMARTS	Tráfico urbano heterogéneo	Entorno 2D/3D enfocado en interacciones sociales	Diseñado específicamente para evaluar algoritmos MARL en escenarios complejos.
VISTA	Conducción por visión (<i>Data-driven</i>)	Fotorrealismo basado en sintetización de datos reales	Generación de trayectorias virtuales a partir de vídeo real sin motor 3D tradicional.
Isaac Sim	Entorno robótico multi-propósito	Fotorrealismo hiperrealista (Omniverse) y físicas PhysX	Simulación acelerada por GPU y sensores de alta fidelidad; muy demandante en hardware.
CARLA	Entorno urbano 3D completo	Fotorrealismo alto (Unreal Engine) y simulación física	Estándar actual para RL/MARL, integración con ROS/SUMO y soporte multisensorial completo.

Estos simuladores pueden clasificarse en dos categorías. Por un lado, los simuladores orientados al flujo de tráfico, como SUMO o SMARTS, priorizan la simulación de un gran número de agentes. Por otro lado, los simuladores orientados al realismo, como CARLA, VISTA o Isaac Sim, buscan reproducir de forma más fiel la percepción y dinámica de vehículos reales mediante sensores realistas.

En este trabajo se ha seleccionado CARLA [4] como entorno debido a varias razones. En primer lugar, proporciona una simulación visual de alta fidelidad basada en Unreal y permite la utilización de sensores como cámaras RGB, segmentación semántica, LiDAR o radar. En segundo lugar, incorpora la posibilidad de modificar las condiciones meteorológicas, incluyendo lluvia, lo que facilita la evaluación de los agentes en las condiciones adversas en las que se pretende desarrollar este proyecto.

Finalmente, CARLA ofrece soporte para escenarios multiagente, permitiendo diseñar entornos adecuados para algoritmos de aprendizaje por refuerzo multiagente. Estas características convierten a CARLA en el simulador más adecuado para el estudio de conducción autónoma MARL donde la ablación perceptual es crítica. Permitiendo captar información mediante cámara RGB, segmentación semántica o radar y proporcionando de forma nativa los datos idealizados del entorno, facilitando la evaluación cualitativa.

4.2. Configuración del entorno de simulación

Una vez seleccionado CARLA como simulador, se procedió a configurar un entorno que permitiera evaluar algoritmos de aprendizaje por refuerzo multiagente en condiciones próximas a un escenario urbano realista con unas condiciones adecuadas para el estudio.

Durante el desarrollo del trabajo se emplearon dos versiones del simulador. Inicialmente, el desarrollo y las primeras pruebas se realizaron sobre CARLA 0.9.14. Posteriormente, debido a falta de recursos al intentar incluir una nueva red para procesar imágenes primero y datos de radar o incluso datos idealizados de CARLA durante el entrenamiento, se usó un contenedor alojado en un servidor proporcionado por la Universidad, servidor que desplegaba en uno de sus puertos el simulador de CARLA con la versión 0.9.16. Todo esto se hizo manteniendo el mismo diseño experimental y verificando la compatibilidad entre ambas versiones.

Como escenario de simulación se seleccionó el mapa *Town10HD*, uno de los entornos urbanos más completos disponibles en CARLA. Este mapa incorpora múltiples carriles, cruces y curvas, proporcionando una elevada diversidad de situaciones de conducción. Además, su nivel de detalle y realismo lo convierten en un entorno ampliamente utilizado para la evaluación de algoritmos de conducción autónoma.

La Figura 4.1 muestra una vista desde arriba del mapa *Town10HD* del simulador CARLA, apreciándose la distribución del trazado vial y la complejidad del entorno.



Figura 4.1: Vista aérea del mapa seleccionado de CARLA [4].

Con el propósito de aproximar el comportamiento del tráfico a un escenario real, en el que unos vehículos pueden estar dentro de lo que se puede denominar 'sistema de agentes' y otros no, se incorporaron vehículos controlados por el sistema (Non-Player Characters, NPCs). Para evaluar la complejidad del entorno y como afecta en la convergencia del algoritmo y la tasa de supervivencia de los agentes, la evaluación se diseña en dos escenarios comparativos:

- **Escenario sin NPCs:** Los agentes interactúan exclusivamente entre sí. Este escenario permite aislar la capacidad para seguir una trayectoria, mantenerse en el carril y evitar colisiones entre agentes sin interferencias externas.
- **Escenario con NPCs :** Se introduce tráfico no cooperativo que circula siguiendo rutas definidas por el piloto automático de CARLA. Este entorno añade un nivel superior de no estacionariedad, obligando a los agentes a reaccionar en tiempo real ante perturbaciones en la calzada, reduciendo el margen de maniobra en condiciones de baja adherencia.

Esta configuración permitirá analizar la degradación del rendimiento de la política aprendida a medida que se incrementa el dinamismo del tráfico circundante.

El estudio se centra en escenarios con condiciones meteorológicas adversas. Para ello, se configuró el simulador utilizando los diferentes parámetros climatológicos mostrados en la Tabla 4.2 que permiten reproducir lluvia intensa, superficies mojadas

y acumulaciones de agua sobre la calzada. Estas condiciones afectan tanto a la percepción como a la dinámica de los vehículos, incrementando la complejidad del problema de navegación.

La configuración experimental considera dos vehículos autónomos controlados mediante aprendizaje por refuerzo multiagente. Esta dimensión facilita el estudio de las dinámicas de coordinación entre agentes al acotar la complejidad durante el entrenamiento.

En el diseño actual del entorno, el espacio de observaciones de cada agente consta de métricas espaciales relativas respecto al otro vehículo del sistema, tales como la posición longitudinal normalizada (Δx_{norm}), la posición lateral normalizada (Δy_{norm}) y la distancia euclídea normalizada (d_{norm}).

Esta definición impone un acoplamiento entre la dimensión del vector de estado y el número de agentes activos. Aunque la arquitectura del algoritmo MAPPO es escalable, extender el entrenamiento a un escenario con $N > 2$ agentes requeriría modificar el estado para evitar un crecimiento lineal de la entrada. Para asegurar una dimensión fija en la red neuronal ante un flujo variable de agentes, sería necesario implementar técnicas como la selección de los k vecinos más próximos (*k-Nearest Neighbors*) o la integración de mecanismos de atención (*Graph Neural Networks*).

Finalmente, cada episodio comienza con los agentes situados en posiciones aleatorias del mapa y un destino objetivo que cada uno debe alcanzar.

La Tabla 4.2 muestra la configuración específica usada durante el desarrollo del proyecto.

Tabla 4.2: Configuración del entorno de simulación en CARLA.

Parámetro	Valor
Versión de CARLA	0.9.16 (desarrollo inicial en 0.9.14)
Mapa	Town10HD
Número de agentes	2
Vehículos NPC	Sí/No(según la prueba a realizar)
Condición meteorológica	Lluvia intensa
Precipitación	80
Acumulación de agua (<i>Precipitation Deposits</i>)	70
Nubosidad	80

4.3. Formulación del problema como Dec-POMDP

El problema que se aborda en este estudio consiste en un aprendizaje cooperativo de múltiples vehículos autónomos que deben desplazarse de forma segura bajo condiciones meteorológicas desfavorables. Debido a que varios agentes toman decisiones de manera simultánea y cada uno de ellos solo dispone de su información particular, siendo algo parcial del entorno, el problema no puede modelarse mediante un *Proceso de Decisión de Markov*, (*MDP*) convencional. En su lugar, se emplea un *Proceso de Decisión de Markov Parcialmente Observable Descentralizado* (*Decentralized Partially Observable Markov Decision Process*, Dec-POMDP), ampliamente utilizado para modelar problemas de aprendizaje por refuerzo multiagente. Formalmente, un Dec-POMDP puede representarse siguiendo [14]:

$$\mathcal{G} = \langle I, S, \{O_i\}, \{A_i\}, P, R, \gamma \rangle$$

donde cada uno de sus elementos representa una característica del problema de decisión.

- **Conjunto de agentes (I).** Representa el conjunto de vehículos autónomos que participan en el entrenamiento. En la configuración desarrollada en este trabajo se consideran inicialmente dos agentes,

$$I = \{1, 2\},$$

aunque la arquitectura implementada permite extender el sistema a un mayor número de vehículos sin modificar el algoritmo de entrenamiento.

- **Estado global (S).** Describe completamente el entorno de simulación, toda la información necesaria para representar la escena en un instante determinado. Durante el entrenamiento, este estado es utilizado exclusivamente por el crítico centralizado siguiendo el paradigma *Centralized Training with Decentralized Execution* (CTDE).
- **Observaciones (O_i).** Cada agente únicamente dispone de una observación parcial del entorno,

$$o_i \in O_i,$$

que incluye aquello que puede percibir localmente. Las observaciones que cada agente realiza se describen en la Sección 3.4.

- **Espacio de acciones (A_i).** Cada vehículo selecciona una acción continua

$$a_i \in A_i,$$

que determina el control aplicado sobre el vehículo. La definición del espacio de acciones empleado se presenta en la Sección 3.5.

- **Función de transición (P).** Modela la probabilidad de transitar desde un estado s hacia un nuevo estado s' cuando todos los agentes ejecutan sus acciones elegidas,

$$P(s'|s, a_1, \dots, a_N).$$

En este estudio, la dinámica viene determinada por el simulador CARLA, encargado de modelar la física del vehículo, las interacciones con el tráfico y la evolución del entorno.

- **Función de recompensa (R).** Evalúa la calidad de las acciones ejecutadas por los agentes. La recompensa incentiva una conducción segura y eficiente, favoreciendo el seguimiento del carril, el avance hacia el destino, la velocidad en un rango seguro y penalizando colisiones y desviaciones laterales y penalizaciones por cercanía a vehículos o peatones. Su formulación detallada se presenta en la Sección 3.6.
- **Factor de descuento (γ).** Determina la importancia relativa de las recompensas futuras frente a las inmediatas,

$$\gamma \in [0, 1].$$

Valores próximos a uno favorecen estrategias con beneficios a largo plazo, mientras que valores bajos priorizan recompensas inmediatas.

Esta formulación permite representar de forma adecuada el problema considerado, en el que múltiples vehículos tratarán de aprender políticas de conducción coordinadas a partir de información parcial del entorno. Sobre esta formulación se

implementa el algoritmo MAPPO, descrito en la siguiente sección, que emplea un entrenamiento centralizado mediante un crítico global y una ejecución descentralizada basada únicamente en observaciones locales.

4.4. Observaciones

Cada agente recibe una observación local compuesta por dos tipos de información: un vector cinemático y de seguimiento de ruta que incluye el estado dinámico del propio vehículo junto a la posición relativa respecto al otro agente, y un vector de percepción del entorno que contiene la proximidad, la alineación lateral y la velocidad respecto a los dos obstáculos más críticos. Con el objetivo de aislar el ruido sensorial y garantizar la estabilidad del entrenamiento, este segundo bloque se calcula mediante información idealizada del simulador aunque durante la evaluación se utiliza el radar para comprobar la exactitud ante la política obtenida. Formalmente, la observación del agente i en el instante t puede expresarse como

$$o_i^t = \{x_i^t, I_i^t\},$$

donde x_i^t representa el vector de estado e I_i^t la percepción del entorno.

4.4.1. Vector de estado

El vector de estado contiene información relativa al propio vehículo, a la ruta planificada y al resto de agentes presentes en el escenario. Todas las variables continuas se normalizan al intervalo $[-1, 1]$ o $[0, 1]$ con el objetivo de facilitar el entrenamiento de la red neuronal.

En este trabajo se emplea el siguiente vector de estado:

$$x_i = [t, \delta, u, v, e_y, e_\psi, e_\psi^{lookahead}, b_f, b_r, \Delta y, \Delta x, d]$$

donde:

- t : posición del acelerador (*throttle*).
- δ : ángulo de dirección (*steering*).
- u : posición del freno (*brake*).

- v : velocidad longitudinal normalizada respecto a la velocidad máxima permitida.
- e_y : error lateral respecto al centro del carril.
- e_ψ : error de orientación respecto al eje del carril.
- $e_\psi^{lookahead}$: error medio de orientación respecto a los tres siguientes *waypoints*, utilizado para proporcionar información sobre la curvatura futura de la trayectoria.
- b_f : componente longitudinal del vector que une el vehículo con el siguiente *waypoint*, proyectada sobre el eje frontal del vehículo.
- b_r : componente lateral del mismo vector proyectada sobre el eje derecho del vehículo.
- Δy : posición relativa lateral respecto al otro agente.
- Δx : posición relativa longitudinal respecto al otro agente.
- d : distancia normalizada entre ambos agentes.

4.4.2. Evolución de la Representación Perceptual y Abstracción Ground Truth

El diseño del espacio de observaciones para la detección de obstáculos e interacción con el entorno, atravesó cuatro fases de evaluación hasta consolidar la arquitectura final empleada durante el entrenamiento:

1. **Entrada visual directa (RGB/Segmentación con CNN):** En las primeras iteraciones se evaluó el uso directo de imágenes RGB y posteriormente máscaras de segmentación semántica utilizando para ellos redes convolucionales (CNN). Esta aproximación incrementó el espacio de observaciones, ralentizando la convergencia de MAPPO y dificultando que la red extrajera información útil de ambas y aislara así la de distancia a los demás vehículos u obstáculos.
2. **Estimación heurística mediante procesamiento matricial (NumPy):** Se intentó derivar las distancias a obstáculos a partir de la imagen de segmentación semántica calculando el área de píxeles pertenecientes a categorías clave (vehículos y peatones). Este enfoque resultó inestable, ya que introducía variaciones no lineales por la perspectiva y, junto a la dificultad de extraer in-

formación simplemente basándose en un área de una imagen de segmentación semántica momentánea, generaba métricas de distancia con alto ruido.

3. **Adopción del Sensor Radar (Representación Escalar):** Se incorporó un sensor Radar frontal (30 m de alcance, 60° FOV). Aunque redujo la dimensión a vectores numéricos (rango, *bearing* y *closing rate*), las lecturas mostraron alta dispersión y fluctuaciones bruscas provocadas por el dinamismo de los NPCs, al introducir ruido ante la imposibilidad de que este sensor aislado, sin información de otros, diferenciase obstáculos estáticos y dinámicos, desestabilizando la actualización de gradientes en el algoritmo MAPPO. No obstante, se decidió corroborar este frente a los datos idealizados durante el proceso de evaluación pretendiendo enfocar una parte del estudio en el ruido introducido cuando tenemos una política ya definida.
4. **Abstracción mediante datos idealizados (Ground Truth):** Ante las limitaciones observadas en los sensores físicos y visuales, se optó por obtener el vector de percepción del entorno directamente a partir de las coordenadas reales proporcionadas por CARLA. Esta estrategia permite aislar el ruido perceptual y evaluar las limitaciones puras de control y de la política en escenarios críticos.

De este modo, el espacio de observaciones de cada agente queda estructurado como un vector denso de 18 dimensiones, según se detalla en la Tabla 4.3.

Tabla 4.3: Variables incluidas en el vector de estado de observación del agente.

Variable	Descripción	Rango
Estado Cinemático y Ruta del Vehículo		
t	Acelerador del paso anterior	$[0, 1]$
δ	Ángulo de dirección del paso anterior (<i>steer</i>)	$[-1, 1]$
u	Freno del paso anterior	$[0, 1]$
v	Velocidad normalizada respecto al máximo (12 m/s)	$[0, 1]$
e_y	Desviación lateral normalizada respecto al centro del carril	$[-1, 1]$
e_ψ	Error angular instantáneo respecto al carril	$[-1, 1]$
$e_\psi^{\text{lookahead}}$	Error angular futuro promedio (próximos 3 waypoints)	$[-1, 1]$

Continúa en la siguiente página...

Continuación de la página anterior

Variable	Descripción	Rango
b_f	Alineación longitudinal hacia el siguiente way-point (<i>bearing forward</i>)	$[-1, 1]$
b_r	Alineación lateral hacia el siguiente waypoint (<i>bearing right</i>)	$[-1, 1]$
Δy	Posición relativa lateral respecto al otro agente	$[-1, 1]$
Δx	Posición relativa longitudinal respecto al otro agente	$[-1, 1]$
d	Distancia euclídea normalizada entre agentes	$[0, 1]$
Percepción del Entorno mediante Ground Truth		
p_1	Proximidad normalizada al obstáculo más cercano	$[0, 1]$
b_1	Desviación lateral en anchos de carril respecto al 1.º obstáculo	$[-1, 1]$
c_1	Tasa de aproximación (<i>closing rate</i>) al 1.º obstáculo	$[-1, 1]$
p_2	Proximidad normalizada al segundo obstáculo más crítico	$[0, 1]$
b_2	Desviación lateral en anchos de carril respecto al 2.º obstáculo	$[-1, 1]$
c_2	Tasa de aproximación (<i>closing rate</i>) al 2.º obstáculo	$[-1, 1]$

Las variables seleccionadas proporcionan al agente información para mantener una velocidad adecuada siguiendo correctamente el carril, la coordinación multi-agente y la anticipación de frenadas. Mientras que las variables cinemáticas y de ruta guían la navegación del vehículo, el bloque de percepción permite inferir riesgos de colisión en entornos de tráfico dinámico (NPCs) o vehículos parados en la calzada.

4.5. Espacio de acciones

Cada agente dispone de un espacio de acciones continuo de dos dimensiones $a_t = [a_{accel}, a_{steer}]^T \in [-1, 1]^2$.

La política generada por la red neuronal produce ambos valores dentro del intervalo $[-1, 1]$. Posteriormente, estas acciones se mapean directamente al formato de control del vehículo en CARLA:

- **Aceleración y Frenado (a_{accel}):** Se utiliza una única variable que divergerá en ambas variables de forma mutuamente excluyente:

$$\begin{cases} throttle = a_{accel}, & brake = 0, & \text{si } a_{accel} > 0 \\ throttle = 0, & brake = |a_{accel}|, & \text{si } a_{accel} \leq 0 \end{cases}$$

Evitando así la aplicación simultánea de acelerador y freno, ya que cualquier valor de frenado que se le pase a CARLA prevalecerá sobre el acelerador.

- **Dirección (a_{steer}):** Se asigna directamente al ángulo del volante $steer = a_{steer} \in [-1, 1]$.

4.6. Función de recompensa

La función de recompensa usada para el aprendizaje de los agentes se base en seguimiento de ruta, estabilidad dinámica, mantenimiento de velocidad y evitación de colisiones mediante el uso del Ground Truth que proporciona CARLA. La recompensa total en cada paso de tiempo se define como:

$$\begin{aligned} R = & R_{heading} + R_{lane} - P_{steer} + R_{progress} + R_{waypoint} \\ & + (R_{speed} + R_{prog_step}) \cdot S_{gate} - P_{obstacle} - P_{low_speed} + R_{terminal} \end{aligned}$$

Error angular ($R_{heading}$) Penaliza las desviaciones de orientación respecto a la carretera:

$$R_{heading} = -2 \frac{|e_\psi|}{\pi/3}$$

Seguimiento y centrado de carril (R_{lane}) Evalúa la desviación lateral del vehículo respecto al centro de la calzada ($w_{lane} = 3$ m), acotando la penalización máxima e incentivando la posición centrada siempre que el vehículo mantenga movimiento:

$$\begin{aligned} R_{lane_raw} &= \text{máx} \left(-4.0, 2.0 - 4.0 \frac{|e_y|}{w_{lane}} \right) \\ R_{lane} &= \begin{cases} R_{lane_raw} \cdot \text{mín} \left(1.0, \frac{v}{2.0} \right) & \text{si } R_{lane_raw} > 0 \\ R_{lane_raw} & \text{en otro caso} \end{cases} \end{aligned}$$

Suavidad de la conducción (P_{steer}) Penaliza las variaciones bruscas en el ángulo de giro entre pasos consecutivos ($\Delta\delta = \delta_t - \delta_{t-1}$) para evitar el zigzag:

$$P_{\text{steer}} = 4.0(\Delta\delta)^2$$

Modulación por riesgo de peligro (S_{gate}) Permite atenuar incentivos de velocidad y progreso cuando existe un obstáculo cercano en la trayectoria. Se calcula a partir de la severidad del riesgo ($D_{\text{severity}} \in [0, 1]$):

$$S_{\text{gate}} = \text{máx}(0.3, 1.0 - 0.7 \cdot D_{\text{severity}})$$

Velocidad objetivo (R_{speed}) e incentivos de avance (R_{progress}) Premia mantener una velocidad próxima al objetivo ($v_{\text{target}} = 8 \text{ m/s}$), modulada por el peligro adyacente S_{gate} :

$$R_{\text{speed}} = 2 \left(1 - \min \left(1, \frac{|v - v_{\text{target}}|}{v_{\text{target}}} \right) \right) \cdot S_{\text{gate}}$$

Adicionalmente, se recompensa con +2 si se supera el récord histórico de cercanía a la meta ($d_{\text{goal}} < d_{\text{min}}$) y +2 (R_{waypoint}) cada vez que el vehículo supera un nuevo *waypoint* de la ruta.

Penalización por obstáculos (P_{obstacle}) y mitigación por frenado Evalúa la proximidad y tasa de aproximación con los dos obstáculos más críticos. La penalización se mitiga proporcionalmente hasta un 80 % si se aplica el freno de forma activa ($u > 0$):

$$P_{\text{obstacle}} = \sum_{k=1}^2 2.5 \cdot \text{danger}_k \cdot (1.0 - 0.8 \cdot u)$$

Penalización por baja velocidad y excusa de parada ($P_{\text{low_speed}}$) Si el agente circula por debajo de 1 m/s durante más de 15 pasos seguidos sin justificación de obstáculos dinámicos(coches o peatones), se aplica una penalización progresiva. Se incluye, no obstante, una "excusa de parada" ($\text{stop_excuse} = \text{mín}(1.0, D_{\text{severity}}/0.3)$) que exime de sanción al agente si este se detiene para evitar una colisión:

$$P_{\text{low_speed}} = 2.0 \cdot \text{máx}(0.0, 1.0 - v) \cdot (1.0 - \text{stop_excuse})$$

Eventos terminales (R_{terminal}) Los episodios finalizan según lo resumido en la Tabla 4.4.

Tabla 4.4: Recompensas, penalizaciones y condiciones de finalización del episodio.

Evento	Condición	Recompensa	Terminal
Alcanzar destino	$d_{\text{goal}} < 2.0$ m	+20	Sí (<i>goal</i>)
Colisión	Impacto detectado por CARLA	−100	Sí (<i>collision</i>)
Desviación de carril	$ e_y > 4.0$ m	−10	No
Salida de ruta	$ e_y > 4.0$ m (> 30 pasos)	−30	Sí (<i>offroad</i>)

4.7. Aprendizaje en el entrenamiento

Durante el proceso de entrenamiento se tiene un crítico centralizado. De esta manera, cada agente tendrá su propia política, y el crítico utilizará información global para estimar el valor de los estados.

El entrenamiento se organiza en epochs de recopilación de experiencias y, tras cada uno, actualización de las redes. Cada epoch consta de un número de steps durante los cuales cada agente interactúa con su entorno y almacena observaciones, acciones y recompensas, además se almacena el valor estimado por el crítico y las probabilidades asociadas a cada acción seleccionada. Esto es precisamente lo que se usa para calcular ventajas y actualizar tanto las políticas de los agentes como el crítico.

4.7.1. Recopilación de experiencias

Durante cada interacción, el agente recibirá una observación compuesta por su estado e información de proximidad a otros vehículos o peatones obtenida mediante el Ground Truth de CARLA. A partir de estas observaciones, cada actor generará los parámetros de una distribución de probabilidad sobre el espacio de acciones, seleccionando esta acción mediante el muestreo. Posteriormente, se aplica esta acción al vehículo de nuevo.

Así, para cada step se almacena, para cada agente, la observación, acción, probabilidad logarítmica de dicha acción y la recompensa. También se guarda el estado global y la estimación de valor que nos proporciona el Critic.

Podemos expresar por tanto un step como:

$$(s_t^i, a_t^i, r_t^i, s_t^{i+1}, \log \pi_{\theta_i}(a_t^i | s_t^i), V_{\phi}(s_t))$$

donde i representa el agente, s_t^i su observación local, a_t^i la acción seleccionada, r_t^i la recompensa obtenida y $V_\phi(s_t)$ la estimación del valor proporcionada por el crítico centralizado.

Las experiencias se recopilarán en un *buffer* hasta alcanzar el número de pasos definido para cada epoch. Una vez completado cada epoch comenzará el proceso de optimización de las redes

4.7.2. Estimación de las ventajas

A partir de las experiencias almacenadas se calcula la ventaja de cada acción, lo que permite determinar si la acción dada ha producido un resultado mejor o peor del que el crítico esperaba. Para esto se ha empleado *Generalized Advantage Estimation* (GAE) [15], combinando así la información de recompensas con las estimaciones de valor futuro.

Yendo paso a paso, primero se calcula el error de la diferencia por cada step (Diferencia Temporal), lo que dice si la acción tomada resultó mejor o peor de lo que el Critic esperaba:

$$\delta_t^i = r_t^i + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

A partir de esto, se obtiene recursivamente la ventaja, que dirá la repercusión a largo plazo de la acción tomada en el presente step:

$$\hat{A}_t^i = \delta_t^i + \gamma \lambda \hat{A}_{t+1}^i$$

donde γ es el factor de descuento y λ controla el compromiso entre sesgo y varianza en la estimación de la ventaja.

Esta recurrencia se consigue recorriendo las experiencia en orden inverso y teniendo en cuenta los estados terminales producidos durante el episodio dado. Tras esto, las ventajas se normalizan buscando estabilidad:

$$\hat{A}_t^i = \frac{\hat{A}_t^i - \mu_A}{\sigma_A + \epsilon}$$

donde μ_A y σ_A representan, respectivamente, la media y la desviación estándar de las ventajas del agente.

Por otro lado, se calcula para el crítico el objetivo, combinando la ventaja estimada y el valor que este ha proporcionado:

$$\hat{V}_t^i = \hat{A}_t^i + V_\phi(s_t)$$

En el sistema desarrollado, los objetivos del crítico se obtienen combinando el promedio de los diferentes agentes, adquiriendo así una referencia común para la actualización de la función de valor centralizada.

4.7.3. Actualización de las políticas

Una vez calculadas las ventajas, cada actor se actualiza utilizando la función objetivo de PPO descrita en anteriores capítulos, reflejando el ascenso por gradiente explicado. La actualización de la política de cada actor i se ejecuta durante $K = 5$ épocas por ciclo de entrenamiento. Para cada acción almacenada se calcula la relación entre la probabilidad asignada por la política actual y la probabilidad con la política anterior lo que ofrecerá así limitar la variación de la política:

$$r_t(\theta_i) = \frac{\pi_{\theta_i}(a_t^i | s_t^i)}{\pi_{\theta_i^{\text{old}}}(a_t^i | s_t^i)}$$

Para evitar inestabilidades numéricas durante el cálculo de este cociente de probabilidad (*ratio*), la diferencia logarítmica de probabilidades se acotó entre $[-10.0, 10.0]$ antes de aplicar la función exponencial:

$$r_t^{(i)}(\theta) = \exp \left(\text{clip} \left(\log \pi_\theta(a_t^{(i)} | s_t^{(i)}) - \log \pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)}), -10.0, 10.0 \right) \right)$$

La función objetivo utilizada corresponde a la variante de PPO, PPO-Clip [16], que aportará más estabilidad al entrenamiento:

$$L^{\text{CLIP}}(\theta_i) = \mathbb{E}_t \left[\min \left(r_t(\theta_i) \hat{A}_t^i, \text{clip} \left(r_t(\theta_i), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t^i \right) \right]$$

Se utiliza un intervalo de clipping de $[0.8, 1.2]$, correspondiente a un valor de $\epsilon = 0.2$. Además, se incorpora un término de entropía que favorece la exploración durante el entrenamiento:

$$L_{\text{actor}} = -L^{\text{CLIP}} - \beta H(\pi_\theta)$$

donde $H(\pi_\theta)$ representa la entropía de la política y β controla la importancia de este término. En la implementación se utiliza $\beta = 0.01$. Tras la retropropagación de L_{actor} , se aplica un recorte de la norma del gradiente a 1.0 antes de actualizar los pesos del actor.

La actualización es independiente para cada uno de los actores, utilizando únicamente las observaciones locales correspondientes a cada agente. Esto permite mantener la ejecución descentralizada característica del paradigma CTDE.

4.7.4. Actualización del crítico

A diferencia de los actores, que optimizan sus políticas de forma descentralizada utilizando únicamente su observación local, el crítico gestiona la parte centralizada de nuestra arquitectura CTDE. Durante la fase de actualización, este recibe el estado global $\mathbf{S}_{\text{global}}$, formado por la concatenación de los estados cinemáticos y los vectores de percepción de los agentes, aprendiendo a estimar el retorno esperado $V_\phi(\mathbf{S}_{\text{global}})$.

Para mejorar la estabilidad y optimizar la estimación de la función de valor, el proceso de entrenamiento del crítico no se realiza en un único bloque. La memoria de experiencias global de tamaño N_{global} se aleatoriza y divide en mini-batches de $M = 256$ a lo largo de $K = 5$ épocas de optimización.

La función de pérdida aplicada sobre cada mini-batch MB se define mediante el Error Cuadrático Medio (MSE) entre el valor estado-valor predicho por la red del crítico V_ϕ y el valor objetivo $\hat{V}_i$ previamente computado mediante GAE:

$$L_{\text{critic}}(\phi) = \frac{1}{|MB|} \sum_{i \in MB} \left(V_\phi(\mathbf{s}_{\text{global},i}, \mathbf{f}_{\text{env_global},i}) - \hat{V}_i \right)^2$$

La optimización de los parámetros ϕ del crítico se ejecuta iterativamente en cada mini-batch mediante la propagación hacia atrás de L_{critic} . Para evitar oscilaciones desestabilizadoras durante la actualización de gradientes, se aplica un recorte de norma de gradiente (*gradient clipping*) acotado a un umbral máximo de 1.0:

$$g_\phi \leftarrow \text{clip}(\nabla_\phi L_{\text{critic}}(\phi), 1.0)$$

Finalmente, los pesos de la red del crítico se actualizan mediante el algoritmo de optimización Adam utilizando el gradiente acotado g_ϕ .

4.7.5. Proceso de actualización

El procedimiento completo puede resumirse entonces como:

1. Los agentes interactúan con CARLA siguiendo sus políticas.
2. Se almacenan las observaciones, acciones, recompensas, probabilidades y valores estimados.
3. Una vez recopilado el conjunto de experiencias, se calculan los errores mediante diferencia temporal δ_t .
4. Se calculan las ventajas mediante GAE y se normalizan.
5. Se obtienen los valores objetivo para entrenar al crítico.
6. Cada Actor se actualiza utilizando la función objetivo PPO-Clip.
7. El Critic centralizado se actualiza utilizando el error cuadrático entre sus predicciones y los valores objetivo.
8. El proceso se repite durante las distintas iteraciones de entrenamiento.

En la implementación además, cada conjunto de experiencias se reutiliza durante cinco epochs antes de recopilar nuevas experiencias, buscando reducir el número de episodios de entrenamiento y estabilizar el aprendizaje. Para evitar actualizaciones excesivamente grandes, se aplica también un límite a la norma del gradiente mediante *gradient clipping*.

Capítulo 5

Solución

5.1. Descripción de la solución

La solución planteada al problema expuesto a lo largo del estudio desemboca en un sistema de conducción autónoma multiagente diseñado para seguir rutas y navegar de forma segura y coordinada en entornos urbanos bajo condiciones meteorológicas de lluvias intensas. Así, el núcleo de todo es el algoritmo de aprendizaje por refuerzo multiagente utilizado MAPPO estructurado en un esquema CTDE.

Para resolver los problemas de inestabilidad que surgen al procesar datos sensoriales como pudieran ser imágenes RGB, mapeos de distancias mediante segmentación semántica o localización de vehículos y peatones mediante radar, se adopta un vector de observaciones basado en el Ground Truth de CARLA. Esto permite extraer de forma categórica el estado relativo de los obstáculos cercanos desde la API.

El desarrollo se integra sobre el mapa Town10HD de CARLA, en el que se incorporan factores de baja adherencia, elementos dinámicos(NPCs) y baja visibilidad. El resultado es un modelo entrenado mediante *Curriculum Learning*, capaz de tomar decisiones en espacios continuos de aceleración y frenado, así como de dirección a partir de su observación local.

5.2. El proceso de desarrollo

5.2.1. Análisis

Definición de requisitos

Para el desarrollo de este sistema de conducción autónoma multiagente se especifican una serie de requisitos funcionales y no funcionales que determinan tanto las capacidades operativas del software como sus restricciones de calidad y estabilidad.

Requisitos funcionales

- **RF-01 (Control multiagente simultáneo):** El sistema debe gestionar el control continuo y coordinado de al menos dos agentes autónomos en un entorno urbano compartido en tiempo real.
- **RF-02 (Arquitectura CTDE y asimetría de información):** La lógica de aprendizaje seguirá la arquitectura *Centralized Training, Decentralized Execution*. La red *Critic* accederá al estado global durante el entrenamiento, siendo este una concatenación del estado los agentes, mientras que la red *Actor* utilizará únicamente el vector de observación local de 18 variables durante la inferencia, 12 de estado propio y 6 referentes a obstáculos circundantes.
- **RF-03 (Entorno y climatología adversa):** El entorno debe simular condiciones de lluvia intensa sobre el mapa Town10HD de CARLA, causando así una reducción en la adherencia.
- **RF-04 (Percepción sintética por Ground Truth):** El módulo de observación debe extraer de la API de CARLA las variables cinemáticas (v, e_y, e_ψ) , revelando así el estado relativo de los obstáculos circundantes.
- **RF-05 (Espacio de acciones continuo):** El modelo debe parametrizar una distribución Gaussiana continua, generando en cada paso acciones de control acotados: dirección $(\delta \in [-1, 1])$ y gas $(g \in [-1, 1])$. Siendo este último separable en acelerador $(t \in [0, 1])$ y freno $(u \in [0, 1])$, que son mutuamente excluyentes.
- **RF-06 (Compartición de parámetros y estabilización de MAPPO):** Las redes de los agentes deben implementar compartición de pesos (*parameter sharing*) en la política del *Actor* para maximizar la eficiencia, junto con normalización de la función de valor para mitigar la no-estacionariedad.

- **RF-07 (Estrategia de Curriculum Learning):** Durante el proceso de entrenamiento debe incorporarse un metodo de inclusión de obstaculos para enseñar a los agentes el frenado en situaciones de excesivo tráfico.
- **RF-08 (Coexistencia con tráfico dinámico):** El entorno debe ser parametrizable para incluir una población fija de 20 vehículos no jugables (NPCs), así como peatones, gestionados por el piloto automático de CARLA, con el fin de evaluar la generalización de la política frente a tráfico de fondo.

Requisitos no funcionales

- **RNF-01 (Tolerancia a fallos del simulador):** Dado que el servidor de CARLA presenta fugas de memoria (*memory leaks*) en ejecuciones prolongadas, el sistema cliente debe implementar un protocolo automatizado de desconexión y reconexión periódica al *host* (cada 3 episodios) sin perder el progreso del entrenamiento.
- **RNF-02 (Estabilidad computacional de la GPU):** El pipeline de entrenamiento debe ejecutar sesiones continuas de hasta 24 horas en GPU sin sufrir desbordamientos de memoria de vídeo (*Out of Memory*), gestionando el vaciado de caché en PyTorch.
- **RNF-03 (Persistencia de datos):** El sistema debe realizar volcados a disco de métricas de forma inmediata para evitar la pérdida de históricos de pérdidas, recompensas y colisiones ante interrupciones no programadas.
- **RNF-04 (Modularidad del software):** El código debe mantener un desacoplamiento estricto entre el cliente de interfaz de CARLA, la definición del entorno estandarizado (Gymnasium) y los módulos de las redes neuronales en PyTorch.

5.2.2. Diseño e Implementación

En esta sección se detalla la arquitectura técnica desarrollada para el sistema de conducción autónoma multiagente, abordando tanto la estructura algorítmica del aprendizaje como las decisiones sobre percepción, representación de estados, funciones de recompensa y mecanismos de tolerancia en el software.

Arquitectura general del sistema MAPPO y paradigma CTDE

El núcleo algorítmico se basa en la adaptación de MAPPO (*Multi-Agent Proximal Policy Optimization*) bajo el paradigma de Entrenamiento Centralizado y Ejecución Descentralizada (*Centralized Training, Decentralized Execution*, CTDE). Este esquema permite lidiar con fluctuación de los entornos multiagente al compartir información durante la fase de entrenamiento y, posteriormente, conservar una ejecución autónoma de cada vehículo durante la inferencia.

- **Actor-Critic:** Evalúa el valor del estado global $V(S)$ al recibir como entrada las observaciones locales y acciones de todos los agentes. Esta red global únicamente se utiliza durante la fase de entrenamiento para guiar la política de los agentes ante un entorno parcialmente observable.
- **Policy:** Define la política de control $\pi_\theta(a_i|o_i)$ para el agente i . Recibe como entrada el vector de observación local o_i de 18 dimensiones, compuesto por 12 dimensiones del estado del vehículo y 6 de los obstáculos más próximos.
- **Parameter Sharing:** Dado que a los agentes controlados por el algoritmo MAPPO se les asigna la misma plantilla (las redes de ambos tienen las mismas capas, bias, etc.), resulta adecuado compartir los pesos en la red *Actor* buscando incrementar la cantidad de muestras y acciones exitosas alcanzadas y acelerando así la convergencia, manteniendo diferencias entre ellos no obstante.
- **Modelado de acciones continuas y acotación de entropía:** La red *Actor* parametriza una distribución Gaussiana estimando la media (μ) y la desviación típica (σ). La media se calcula mediante una función de activación **tanh** reescalada y desplazada según la escala y el *offset* específicos de las acciones. Por su parte, la desviación típica se obtiene a partir de **log_std**, aplicando una acotación inferior (**log_std_floor**) para garantizar una exploración mínima continua y un límite superior ($\sigma \leq 1.0$) mediante **clamp** para evitar la inestabilidad por ruido excesivo en la política.

Evolución del modelo de percepción

La selección del modelo de percepción fue el resultado de un proceso iterativo de experimentación. En las primeras etapas del proyecto se evaluaron sensores visuales (cámara RGB y segmentación semántica), los cuales fueron descartados progresivamente debido a imposibilidades de convergencia o a desatender la información proporcionada por no tener valor. En la Tabla 5.1 se resumen las etapas de esta evolución técnica.

Tabla 5.1: Evolución iterativa del sistema de percepción y justificación de decisiones.

Modalidad	Problema / Limitación	Decisión de Ingeniería
Cámara RGB	Alto coste computacional (CNNs), deslumbramientos y demasiada información inútil que la red no sabía cómo procesar.	Descartado: Inestabilidad en la convergencia del Actor.
Segmentación Semántica	Renderizado lento en CARLA, sensibilidad a oclusiones e información compleja para la red.	Descartado: Provocaba cuellos de botella e inestabilidad.
Distancia en píxeles	Inviabilidad geométrica para inferir distancias desde el área en píxeles. La proyección impide estimar rangos con precisión.	Descartado: Mediciones erráticas que imposibilitaban el frenado.
Radar	Dificultad para separar un objeto dinámico de uno estático, generando episodios de búsqueda de objetos para obtener recompensa.	Descartado: Incapaz de mantener una conducción constante.
Ground Truth	Ninguno. Lectura directa de variables y obtención de distancias desde la API del simulador.	Seleccionado: Determinista, ligero (18 variables) y robusto frente al clima.

Se debe mencionar que, una vez garantizada la convergencia de la política mediante el *Ground Truth* de CARLA, se plantea como fase experimental la reincorporación del sensor Radar durante la evaluación. Se busca observar el comportamiento del sistema al filtrar y contrastar las detecciones del radar directamente contra estos datos ideales y compararla con su utilización de forma aislada. El propósito es determinar si un esquema de validación cruzada permite mitigar la dispersión evaluando la viabilidad de recuperar observaciones sensoriales más realistas sin desestabilizar la política aprendida.

Definición del espacio de estados y acciones

El estado de cada agente queda determinado por un vector de 18 variables continuas extraídas en tiempo real desde la API de CARLA, eliminando la latencia del renderizado gráfico.

Tabla 5.2: Composición del vector de observación local o_i (18 variables).

Categoría	Dim.	Descripción de las variables
Estado de control previo	3	Valores aplicados en el paso anterior para acelerador (t), dirección (δ) y freno (u).
Cinemática	1	Velocidad escalar normalizada ($v/v_{\max}$).
Geometría y seguimiento de ruta	5	Desviación lateral al carril (e_y), error angular local (e_1), error angular medio a 3 <i>waypoints</i> (e_2), y proyecciones del vector al siguiente <i>waypoint</i> ($\text{bearing}_{\text{fwd}}$, $\text{bearing}_{\text{right}}$).
Coordinación inter-agente	3	Posición relativa normalizada en X e Y ($\Delta x, \Delta y$) y distancia euclídea normalizada respecto al otro agente ego coordinado.
Detección de obstáculos (GT / Radar)	6	Proximidad, desviación lateral relativa al carril y velocidad de aproximación (<i>closing rate</i>) de los 2 obstáculos más críticos (NPCs, peatones u otros agentes).

Así, el vector de observación concatena el bloque **vehicle_state** (12 dimensiones) y el bloque **cam_features** (6 dimensiones). El primer bloque registra las acciones previas del vehículo, su velocidad escalar normalizada, las métricas de seguimiento de trayectoria y las distancias relativas de coordinación con el otro agente.

Por su parte, el bloque de percepción analiza los vehículos NPCs y los peatones situados en un cono frontal de aproximadamente 80° . Tras filtrar los elementos fuera de la ruta o en carriles adyacentes no relevantes, evalúa y ordena los obstáculos según su nivel de amenaza, extrayendo de los dos elementos más críticos su distancia, su desviación lateral y su tasa de aproximación.

El espacio de acciones es continuo y tridimensional, definiendo el control físico directo del vehículo: $a = [t, \delta, u]$, donde el acelerador se acota en $t \in [0, 1]$, el ángulo de dirección en $\delta \in [-1, 1]$ y el freno en $u \in [0, 1]$.

Diseño de la función de recompensa y Curriculum Learning

El diseño del sistema de incentivos y el esquema de aprendizaje mediante currículo constituyen el núcleo para dirigir la convergencia del algoritmo MAPPO buscando evitar la detección indefinida, la alta velocidad y buscando la conducción centrada y fluida.

Formulación de la Recompensa

La función de recompensa R_t acumulada en cada paso de simulación se compone de términos de progreso, de penalización por peligro y de penalizaciones terminales:

$$R_t = R_{\text{alineación}} + R_{\text{centrado}} + R_{\text{suavidad}} + R_{\text{progreso}} + R_{\text{velocidad}} - P_{\text{peligro}} - P_{\text{parada}} + R_{\text{terminal}}$$

- **Alineación y Centrado Lateral:** Se penaliza el error angular e_θ respecto a la vía y la desviación del centro del carril e_y (ancho de carril $w = 3$ m):

$$R_{\text{alineación}} = -2 \cdot \frac{|e_\theta|}{\pi/3}, \quad R_{\text{centrado}} = \max\left(-4.0, 2.0 - 4.0 \cdot \frac{|e_y|}{w}\right) \cdot \min\left(1.0, \frac{v}{2.0}\right)$$

- **Suavidad de la Acción:** Para evitar (*zig-zag*), se aplica un castigo cuadrático sobre la variación del volante en pasos simultáneos: $R_{\text{suavidad}} = -4.0 \cdot (\Delta a_{\text{steer}})^2$.
- **Modulación por Seguridad (*Speed Gate*):** Las recompensas por avance (Δd_{meta}) y por aproximación a la velocidad objetivo ($v_{\text{target}} = 8$ m/s) quedan condicionadas por la severidad del peligro detectado ($D_{\text{severidad}} \in [0, 1]$):

$$\text{speed_gate} = \max(0.3, 1.0 - 0.7 \cdot D_{\text{severidad}})$$

$$R_{\text{progreso}} = \Delta d_{\text{meta}} \cdot \text{speed_gate}$$

$$R_{\text{velocidad}} = 2.0 \cdot \left(1 - \min\left(1, \frac{|v - v_{\text{target}}|}{v_{\text{target}}}\right)\right) \cdot \text{speed_gate}$$

- **Penalización por Peligro y Frenado Mitigado:** Ante la presencia de obstáculos detectados en el Ground Truth, la penalización se atenúa si el agente está aplicando el freno (a_{freno}):

$$P_{\text{peligro}} = \sum_{k \in \{\text{obs}\}} \gamma_{\text{prox}} \cdot \text{Peligro}_k \cdot (1.0 - 0.8 \cdot a_{\text{freno}})$$

- **Parada Excusada (*Stop Excuse*):** Si la velocidad cae por debajo de 1.0 m/s durante más de 15 steps consecutivos, se penaliza salvo que exista un peligro que lo justifique ($\text{stop_excuse} = \min(1.0, D_{\text{severidad}}/0.3)$):

$$P_{\text{parada}} = 2.0 \cdot \max(0.0, 1.0 - v) \cdot (1.0 - \text{stop_excuse})$$

- **Recompensas Terminales y Bonificaciones:** Se otorgan recompensas por reducción de distancia récord (+2), superación de waypoints (+2) y llegada a la meta (+20). Las condiciones de finalización ($\text{done} = \text{True}$) con castigo incluyen colisión (-100) y invasión prolongada fuera de ruta ($|e_y| > 4$ m por 30 pasos, -30).

Curriculum de Colisión Dinámico (Obstacle Injection & Release)

Para forzar al agente a aprender maniobras de frenado sin depender de encuentros casuales con el tráfico aleatorio de CARLA, se implementó un mecanismo de *Curriculum de Colisión*:

1. **Inyección de Obstáculo:** Al reiniciar el episodio, se posiciona un vehículo NPC el 70 % de las veces detenido directamente en la trayectoria trazada por el planificador, a una distancia de entre 20 m y 35 m por delante del agente.
2. **Criterio de Liberación (*Release*):** Si el agente reduce su velocidad ($v < 0.5 \text{ m/s}$) y mantiene una detención segura frente al obstáculo sin colisionar durante un tiempo acumulado $t_{\text{espera}} \in [4, 10]$ segundos, se da el problema como resuelto y se activa el piloto automático del NPC para reanudar la marcha y despejar la vía.
3. **Reenganche (*Rearm*):** Tras recorrer 10 waypoints adicionales sin incidentes, el sistema vuelve a evaluar la inyección de un nuevo obstáculo en la ruta.

Mecanismos de robustez técnica y gestión del simulador

Para superar las limitaciones operativas de CARLA en ejecuciones prolongadas de entrenamiento, se diseñó una capa de infraestructura en Python con los siguientes mecanismos:

- **Gestión de fugas de memoria y reconexión automatizada:** Se programó un reinicio de la conexión al host de CARLA cada 3 episodios. Esto libera la memoria RAM/VRAM retenida por los procesos internos de CARLA y evita cierres colapsos por memory leaks reportados de CARLA.
- **Liberación activa de caché en GPU:** Tras cada actualización de gradientes en PyTorch, se invoca explícitamente a `torch.cuda.empty_cache()` y al recolector de basura de Python (`gc.collect()`).
- **Gestión de colisiones en el *Spawn*:** Se implementó una lógica de reintento para la reaparición de agentes, evitando situar a los agentes si por esos puntos estaba circulando algún NPC, ya que provocaba episodios enteros de colisión al caer sobre estos.
- **Escritura inmediata de logs:** El volcado de métricas de entrenamiento a disco se realiza de manera síncrona en cada paso, garantizando la persistencia de los datos ante posibles caídas del sistema.

Capítulo 6

Evaluación

6.1. Proceso de entrenamiento

El proceso de entrenamiento consistió en entrenar dos agentes mediante MAPPO con actores independientes y crítico centralizado. Se configuró el mapa Town10HD con lluvias y depósitos de agua. Además de 20 vehículos NPC y 20 peatones.

Se incluyó el explicado a lo largo de todo el Trabajo de Fin de Master, *Curriculum Learning* dinámico, diseñado para escalonar el aprendizaje de evitación de obstáculos ante las pocas muestras que obtendríamos sin el mismo.

Este entrenamiento se extendió durante un total de 465 episodios de simulación, donde cada episodio se compone de 2048 pasos. Tras alcanzar dicho episodio, se finalizó el entrenamiento al observarse un claro aplanamiento en lo referente a llegadas a la meta, evitación de obstáculos y velocidad media entre otras.

A continuación, se recogen y analizan los datos cuantitativos y las métricas obtenidas a lo largo de este periodo de aprendizaje.

6.1.1. Definición de métricas y limitaciones

Antes de proseguir, se debe explicar que significan las métricas que se van a discutir durante esta sección:

- **Terminaciones:** Cada colisión producida se contabilizará como una terminación, a pesar de que el episodio siga desarrollándose con normalidad al respawnear el agente en el inicio de la ruta.

- **Colisión sin velocidad:** La interpretabilidad de las colisiones debe ir relacionada con la velocidad en que se producen las mismas. Es decir, no se puede interpretar que un episodio es mejor por producirse en él menos colisiones, si la causa raíz de estas menores colisiones es la estaticidad del agente.
- **Progreso:** El avance en la ruta a lo largo de un episodio representa una métrica valiosa. Un rollout sin terminación no significa ausencia de progreso en el entrenamiento.
- **Intervalo de confianza(IC):** Las llegadas a la meta son sucesos poco frecuentes, por lo que su tasa se acompañará del Intervalo de Confianza de Poisson al 95 %. Dos ventanas cuyos intervalos se solapan no seán distinguibles.

6.1.2. Estabilidad de la optimización

La Figura 6.1 verifica que el proceso de optimización es estable antes de que se pase a analizar el comportamiento resultante.

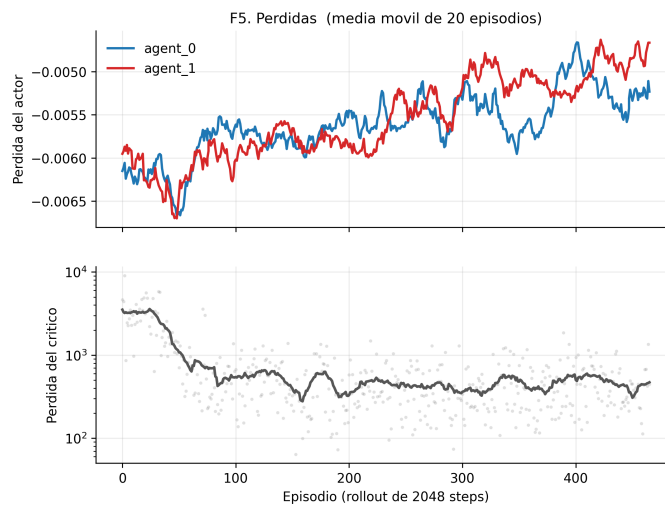


Figura 6.1: Función de pérdidas de los actores y el crítico

- **Critic Loss(panel inferior a escala logarítmica):** La pérdida del crítico se aprecia totalmente estable en la banda 400-600 tras descender claramente en los primeros cien episodios de 3500 a 500. Esto nos dice que el crítico aprende rápidamente la escala y, posteriormente, se queda limitado por la varianza intrínseca del entorno. No pudiendo otorgar esta limitación a la capacidad de la red.
- **Actor Loss(panel superior):** Se mantiene en valores muy pequeños, entre -0.0065 y -0.0050, con una tendencia creciente en todo caso. Por supuesto, se

debe señalar que el término de la entropía (cuyo coeficiente es 0.01) domina esta magnitud, por lo que su valor absoluto no debe interpretarse como calidad de la política. La relevancia de esta gráfica es que no diverge en ningún momento, poniendo de manifiesto que el recorte de PPO y la normalización de las ventajas, mantienen acotadas las actualizaciones. El crecimiento es así coherente con la reducción de la entropía mientras que la desviación típica se aproxima a su cota inferior.

6.1.3. Evolución de la recompensa

La Figura 6.2 se aprecia la evolución de la función de recompensa a lo largo de los episodios.

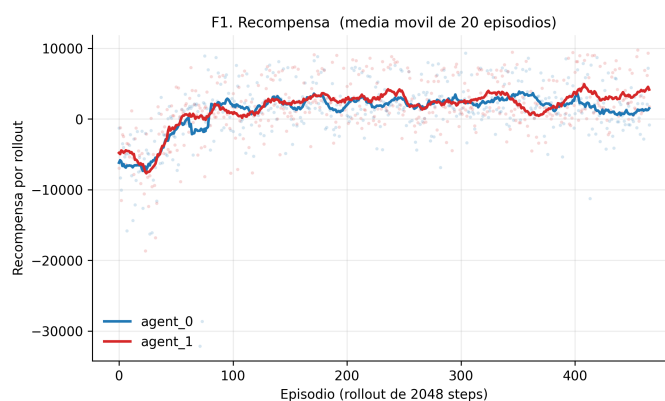


Figura 6.2: Función de recompensa

Se puede distinguir como la recompensa media pasa de valores negativos en los primeros cien episodios (-2548 y -2062 respectivamente para el agent_0 y el agent_1) a valores positivos y estables a partir de ese punto. Más relevante es, si cabe, la reducción de la dispersión; se observa como la desviación típica cae de 6484 a en torno 3000 para el agent_0, lo que constata un comportamiento más firme además de mejor.

Sin embargo, tras este hito, la recompensa se estanca, oscilando entre los episodios 100 y 465 sin ninguna tendencia clara. Esto ilustra lo mencionado con anterioridad referente a usar la recompensa por si sola como indicador de progreso.

Así, la recompensa se utiliza para medir la adhesión a la conducción pero no el cumplimiento de la tarea. Por ello, la siguiente métrica clarificarán mejor el comportamiento visibilizado.

6.1.4. Progreso hacia el destino

La siguiente Figura 6.3 muestra la fracción de ruta completada por los agentes.

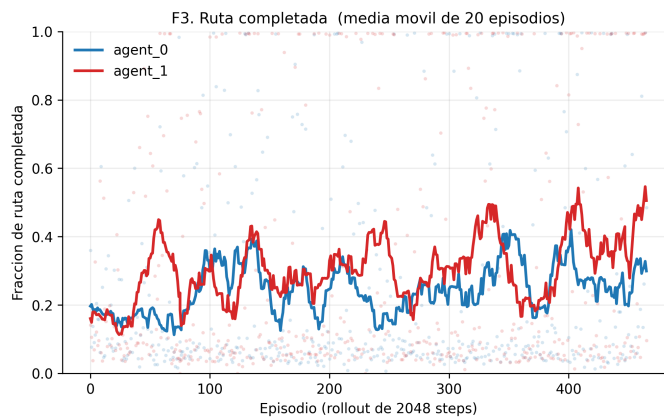


Figura 6.3: Fracción de ruta completada

En ella se aprecia como el agent_1 una progresión de forma sostenida. En contraparte, el agent_0 mejora inicialmente para estabilizarse a partir de ese momento.

Por su parte, en la Figura 6.4 se hace visible la evolución acumulada de llegadas a la meta fijada en el inicio de cada episodio.

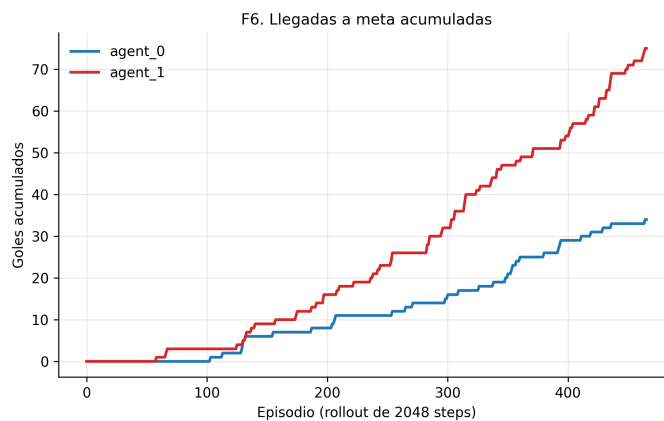


Figura 6.4: Llegadas a meta acumuladas

Las curvas exhiben convexidad creciente; la pendiente del tramo final supera claramente a la de los primeros episodios, confirmando de esta manera que la tasa de éxito aumenta y no simplemente acumula sucesos a ritmo constante. Al término del entrenamiento, el agent_1 acumula 75 llegadas a meta frente a las 34 del agent_0. En ambos agentes se aprecia la mejora neta. Multiplicando por más de 10 la tasa de llegadas.

Estas figuras son las más informativas, por mostrar el cumplimiento efectivo de la tarea.

6.1.5. Seguridad de conducción

A continuación se muestra la Figura 6.5 que expresa el número de colisiones por episodio.

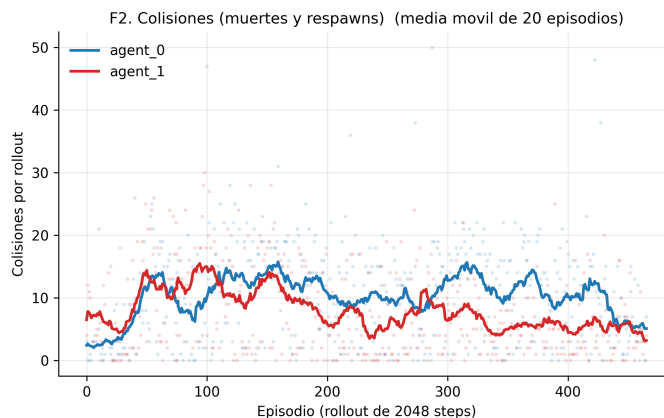


Figura 6.5: Colisiones por episodio

En ella se revela un comportamiento opuesto del `agent_1` y el `agent_0`. Mientras que el primero reduce sus colisiones de forma sostenida a más de la mitad, el segundo describe un máximo en trono al episodio 310 y acaba descendiendo significativamente al final del entrenamiento.

No se puede sintetizar una conclusión de esto sin tener en cuenta la Figura 6.6. En ella se termina de dar contexto a la figura anterior, al mostrar como la velocidad media se duplica durante los primeros doscientos episodios para mantenerse oscilante en torno a 3.5 y 4.5 m/s a partir de ese momento. Esto va a reflejar que la reducción del número de colisiones no se debe a la estaticidad de los agentes si no a su mejora en la conducción. Para resaltar esto, si se observan los primeros 50 episodios, la tasa de colisiones es baja pero va de la mano con una gran estaticidad de los agentes.

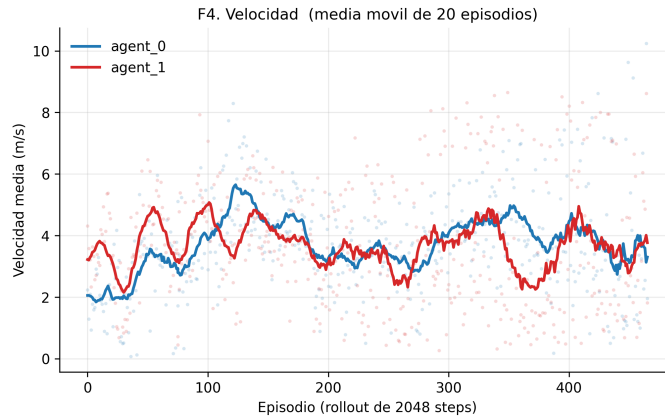


Figura 6.6: Velocidad Media de los agentes por episodio

6.1.6. Síntesis cuantitativa

La tabla 6.1 consolida lo mostrado en las gráficas anteriores. El estudio de la evolución se estudia en ventanas de 50 episodios por agente. Se incorporará en ella el Intervalo de Confianza sobre la tasa de llegadas y el número de observaciones válidas empleadas.

Tabla 6.1: Evolución del entrenamiento en ventanas de episodios (2048 *steps* con *respawn*). Intervalo de confianza al 95 % (Poisson).

Ventana	<i>n</i>	Goles	Goles/ ep	IC 95 %	Colis.	Veloc.	Ruta Media	%Ruta > 0.30	<i>n</i> ruta
Agente 0 (<i>agent_0</i>)									
ep0-49	50	0	0.00	[0.00, 0.00]	4.4	2.24	0.175	18 %	49
ep50-96	50	0	0.00	[0.00, 0.00]	10.1	3.22	0.174	17 %	47
ep97-146	50	6	0.12	[0.02, 0.22]	12.9	4.82	0.329	44 %	50
ep147-196	50	2	0.04	[0.00, 0.10]	13.1	4.10	0.206	20 %	49
ep197-246	50	3	0.06	[0.00, 0.13]	10.2	3.41	0.241	30 %	50
ep247-296	50	3	0.06	[0.00, 0.13]	9.6	3.25	0.197	17 %	48
ep297-346	50	5	0.10	[0.01, 0.19]	12.6	4.38	0.270	28 %	50
ep347-396	50	10	0.20	[0.08, 0.32]	10.9	4.26	0.318	30 %	50
ep397-446	50	4	0.08	[0.00, 0.16]	9.9	3.65	0.250	25 %	48
ep447-465	20	1	0.05	[0.00, 0.15]	5.5	3.87	0.319	32 %	19
Agente 1 (<i>agent_1</i>)									
ep0-49	50	0	0.00	[0.00, 0.00]	7.4	3.22	0.185	20 %	50
ep50-96	50	3	0.06	[0.00, 0.13]	12.0	4.16	0.300	37 %	49
ep97-146	50	6	0.12	[0.02, 0.22]	11.5	4.23	0.287	29 %	49
ep147-196	50	5	0.10	[0.01, 0.19]	10.6	3.69	0.250	22 %	50
ep197-246	50	9	0.18	[0.06, 0.30]	5.9	3.53	0.370	37 %	46

Continúa en la siguiente página...

Continuación de la tabla anterior

Ventana	n	Goles	Goles/ ep	IC 95 %	Colis.	Veloc.	Ruta Media	%Ruta > 0.30	n ruta
ep247-296	50	9	0.18	[0.06, 0.30]	7.9	3.32	0.276	30 %	46
ep297-346	50	15	0.30	[0.15, 0.45]	6.5	4.20	0.373	38 %	50
ep347-396	50	6	0.12	[0.02, 0.22]	5.9	2.74	0.226	20 %	49
ep397-446	50	16	0.32	[0.16, 0.48]	5.1	3.94	0.411	42 %	48
ep447-465	20	6	0.30	[0.06, 0.54]	4.7	3.55	0.449	40 %	20

La tabla evidencia que para el `agent_1` la tasa de llegadas presenta oscilaciones que no son distinguibles del ruido, ya que los intervalos de ventanas consecutivas se solapan. Resulta así más fiable la métrica de una observación por episodio para este estudio, dónde se evidencia que las colisiones se reducen casi un tercio y la ruta completada asciende 4 veces más.

6.1.7. Asimetría entre agentes y Limitaciones

Pese a compartir arquitectura, hiperparámetros y Crític. El comportamiento de los dos agentes divergen de forma clamorosa. En los 470 episodios, `agent_1` acumula 75 llegadas a la meta frente a 34 del `agent_0`. También colisiona 8.0 veces por episodio este primero frente a 10.2.

Más relevante que la magnitud es la naturaleza distinta de lo aprendido, visible al contrastar las Figuras 6.5, 6.3, 6.4. El `agent_1` mejora simultáneamente en progreso y seguridad, mientras que el `agent_0` mejora únicamente en progreso.

Una explicación reside en la estructura del crítico centralizado, apreciable también en la forma de sierra de las distintas figuras mostradas. El objetivo de valor se calcula como la media de los retornos de ambos agentes sobre un estado global, por lo que el crítico no puede atribuir el valor a un agente concreto y la mejora de uno desplaza la línea base del otro.

Por otro lado, las métricas de entrenamiento incorporan el ruido de exploración de la política estocástica, cuya desviación típica está acotada inferiormente durante el entrenamiento; por tanto la tasa de fallo está aumentada respecto a la política determinista evaluada en el capítulo siguiente.

6.2. Proceso de evaluación

Se utiliza un único conjunto de pesos tanto para validaciones con NPCs como sin NPCs, ya que durante este Trabajo de Fin de Master se busca la creación de un

modelo capaz de generalizar a cualquier situación. Un entrenamiento sin obstáculos facilitaría el aprendizaje del seguimiento de ruta pero no sería extrapolable a ninguna otra situación. Por ende, este modelo es el entrenado con *Curriculum Learning* y tráfico denso. La evaluación se realiza en cuatro escenarios independientes:

- **Escenario sin NPCs:** Evaluación de capacidades puras de navegación y centrado de carril en mapa libre. Situación que nos permitirá ver la generalización, al no haberse entrenado nunca en mapa despejado de obstáculos.
- **Escenario con NPCs y Ground Truth de CARLA:** Evaluación de capacidades de evasión dinámica y frenado de emergencia ante obstáculos dinámicos.
- **Escenario con NPCs contrastando el radar con el Ground Truth:** Ampliación del escenario anterior en el que los datos serán los del radar contrastados primero con los idealizados de CARLA.
- **Escenario con NPCs y radar bruto:** Evaluación de una posible circulación con datos brutos obtenidos por el radar.

Se evaluó con acciones deterministas, usando la media de la distribución sin ruido, sobre 30 episodios por condición.

6.2.1. Rendimiento con percepción idealizada de CARLA

En la Tabla 6.2 se muestran los resultados obtenidos por agente y por escenario.

Tabla 6.2: Resultados de evaluación por escenario y agente, con acciones deterministas (media de la gaussiana). Cada episodio aporta un único desenlace.

Agente	n	% Meta	% Colis.	% Fuera ruta	% Time-out	Recomp.	Vel. media	Vel. máx.	Ruta	Steps
Sin tráfico (<i>no_traffic</i>)										
<i>agent_0</i>	30	43.3 %	53.3 %	3.3 %	0.0 %	1948	11.92	18.03	0.848	514
<i>agent_1</i>	30	60.0 %	40.0 %	0.0 %	0.0 %	3157	9.86	11.89	0.829	598
Ambos	60	51.7 %	46.7 %	1.7 %	0.0 %	2552	10.89	14.96	0.839	556
Con tráfico (<i>traffic</i>)										
<i>agent_0</i>	30	16.7 %	80.0 %	3.3 %	0.0 %	1403	8.83	14.13	0.455	390
<i>agent_1</i>	30	30.0 %	70.0 %	0.0 %	0.0 %	3531	7.55	10.77	0.666	715
Ambos	60	23.3 %	75.0 %	1.7 %	0.0 %	2467	8.19	12.45	0.560	552

Sobre el mapa despejado se identifica como la política completa la ruta en más de la mitad de los episodios y, además, recorre de media el 83.9 % del trayecto. Esto pone de manifiesto que el control base es totalmente funcional.

La introducción de tráfico reduce esta tasa de éxito a menos de la mitad y la fracción completada la reduce un tercio frente a la carretera despejada. Esto deja fehaciente el coste de la presencia de otros agentes. Cuando surge la necesidad de anticipar y ceder, la calidad de conducción se reduce significativamente.

Nuevamente, cabría destacar que la métrica mas informativa es, realmente, la fracción de ruta completada, ya que el hecho de llegar al objetivo es un binario donde no sabemos como de bien se ha actuado.

En la Figura 6.7 se aprecia esta binarización que cuenta las colisiones y llegadas a meta en los diferentes escenarios expuestos

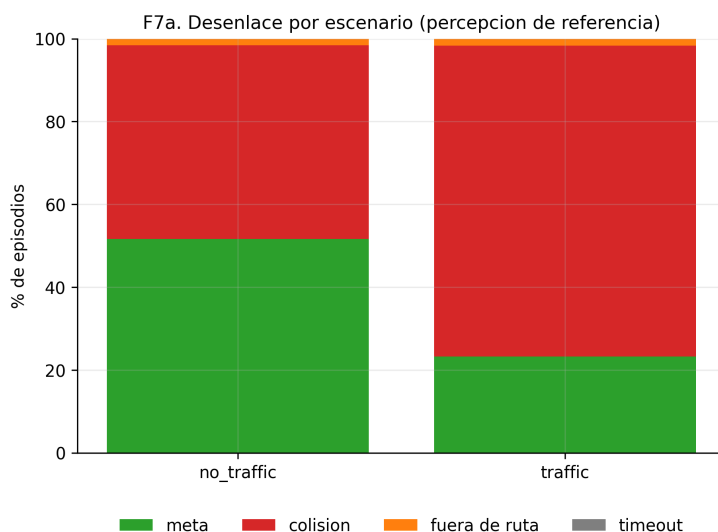


Figura 6.7: Tasa de llegadas a meta y colisiones

Mientras en la Figura 6.8 la información proporcionada es mucho más rica, apreciándose como sin tráfico solo un 11.7% de las veces no llega a la mitad del recorrido, mientras con tráfico se multiplica por cuatro, el 48.3% de los episodios no llega ni a la mitad del recorrido.

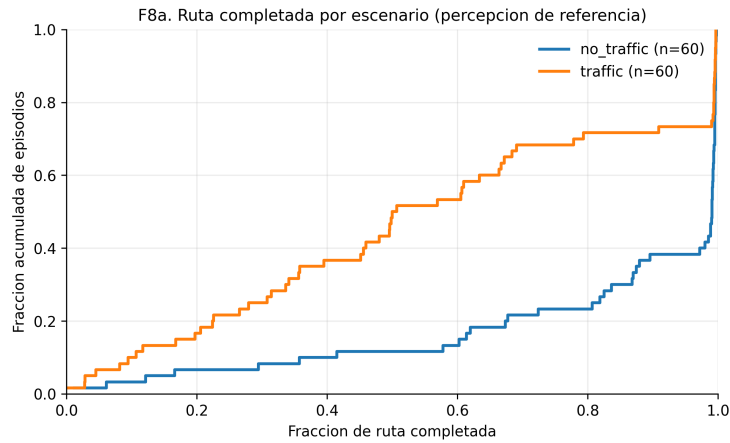


Figura 6.8: Distribución de ruta completada

Además se aprecia como sin tráfico el comportamiento es casi binario, o recorre toda la ruta o falla pronto. Por otro lado, el comportamiento con tráfico es mas lineal, los fallos ocurren en cualquier punto del trayecto.

6.2.2. Ablación del stack de percepción

En la siguiente Tabla 6.3 se resaltan los datos que expresan lo comentado durante el presente estudio y que demuestra porque se decidió usar el Ground Truth que proporciona la API, al mostrar los resultados catastróficos de usar la información del radar bruto.

Tabla 6.3: Ablación del *stack* de percepción en el escenario con tráfico. La política y los pesos son idénticos en las tres filas: solo cambia el origen de las *features*. Las columnas Δ representan la diferencia frente al *ground truth*.

Percepción	n	% Meta	% Col.	% Fuera ruta	% Time-out	Recomp.	Vel. med.	Vel. máx.	Ruta	Steps	Δ % Meta	Δ % Col.	Δ Ruta
Ground truth	60	23.3 %	75.0 %	1.7 %	0.0 %	2467	8.19	12.45	0.560	552	0.0	0.0	0.000
Radar validado	60	33.3 %	63.3 %	0.0 %	3.3 %	2763	7.49	12.97	0.619	745	+10.0	-11.7	+0.059
Radar crudo	60	0.0 %	10.0 %	3.3 %	86.7 %	967	0.67	2.94	0.136	1878	-23.3	-65.0	-0.424

Si el radar se valida con los datos ideales, no se aprecia diferencia detectable, ya que ambas condiciones resultan estadísticamente indistinguibles. Sus Intervalos de Confianza se solapan: [12.6 %, 34.0 %] y [21.4 %, 45.3 %] respectivamente. La diferencia de 10 puntos a favor del radar validado por llegada a la meta se sitúa muy por debajo de la resolución del experimento, porque puede proceder de variantes del propio sistema no concluyentes.

Este resultado, lejos de ser sorprendente, es coherente con la construcción. `radar_validated` no es un sistema de percepción autónomo; es un híbrido en el que, aunque la geometría proceda del sensor, la clasificación procede del simulador. Esa fuente no existe fuera de CARLA. En un vehículo real no hay ninguna lista de actores que consultar. La condición `radar_raw` mide exactamente lo que ocurre cuando se retira esta idealización.

Con los datos del radar brutos se obtiene una caída de la tasa de éxito al 0 %; el vehículo queda prácticamente detenido, a 0.67 m/s. El contraste entre ambas condiciones aísla así la clasificación como el componente responsable del fallo. El radar debe entenderse como una fuente de geometría fiable pero semánticamente ciega, y requiere obligatoriamente un canal complementario que aporte la información que en este experimento suministraba el simulador, como podría ser segmentación semántica.

Se muestra en la Imagen 6.9 de una manera más visual, como se hizo en el apartado anterior la ruta completada según, ahora, el sistema de percepción usado.

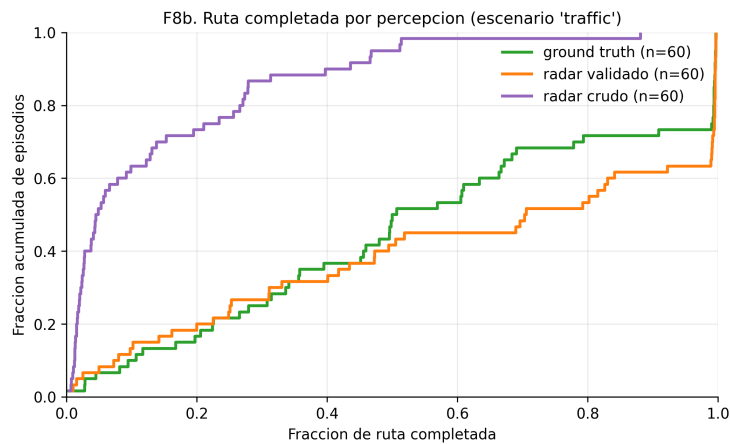


Figura 6.9: Distribución de ruta completada con datos brutos del radar

6.2.3. El embudo de la detección

La tabla 6.4 muestra el colapso tratado en la sección anterior

Tabla 6.4: Embudo de procesamiento del radar por *tick*. El *clustering* agrupa las múltiples reflexiones de un mismo objeto; la validación contra *ground truth* descarta las que no corresponden a ningún actor. El porcentaje final indica cuánta de la señal cruda es entorno estático.

Percepción	Ticks	Detecciones/ tick	Clusters/ tick	Validados/ tick	Usados/ tick	% Clusters validados
Radar crudo	121 210	42.8	16.25	0.46	6.98	2.8 %
Radar validado	62 772	55.8	14.46	1.05	0.75	7.3 %

Se observa como el agrupamiento en cúlsters funciona, al comprimir unas 50 reflexiones por tick en unos 14-16 objetos. Pero la diferencia decisiva está en usados, donde el radar crudo entrega a la política 6.98 obstaculos por ticks fente a 0.74 del validado. Y solo el 2.8 % de los clústers corresponde a un actor real. Es decir, con el radar bruto, la señal de peligro está permanentemente activa.

6.2.4. Separabilidad de la señal de radar

Porcede entonces preguntar si la información entregada por el sensor permitiría construir el clasificador que no posee el radar.

Se evaluó así la velocidad radial, que podría ser la única candidata de entre los datos ofrecidos por el radar para discriminar objetos estáticos de dinámicos. Para esta evaluación, se hizo uso del AUC y el coeficiente de solapamiento como se refleja en la tabla 6.5

Tabla 6.5: Separabilidad por velocidad radial calculada de forma independiente en cada condición. La velocidad radial de un objeto estático coincide con la del propio vehículo.

Agregar ambas poblaciones cancela sus sesgos y produce un valor espurio.

Condición	n actor real	n estático	AUC vel. radial	Solapamiento	Media vel. actor	Media vel. estático
Radar crudo	2 812	96 016	0.645	0.707	2.017	0.578
Radar validado	3 268	42 364	0.293	0.606	3.161	6.549

La tabla muestra la imposibilidad de la velocidad radial para esta solución planteada y la causa es puramente física y confirma la medida. La velocidad radial de un objeto estático coincide con la del propio vehículo. De ello se desprende la conclusión central: no existe un umbral fijo sobre la velocidad radial capaz de separar actores de entorno estático, porque el valor que caracteriza a lo estático es la velocidad del propio observador y varía continuamente. La Figura 6.10 nos facilita la distinción comentada.

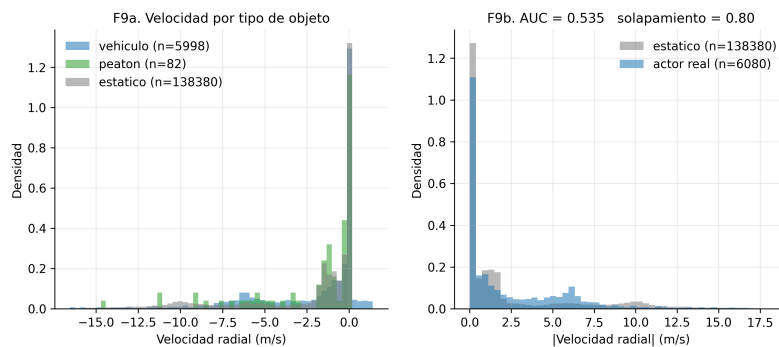


Figura 6.10: Velocidad radial de los objetos detectados para diferenciar por esta

El 84.4 % de las detecciones tienen signo negativo, lo que significa que el objeto se detecta aproximándose. Es lo esperable ya que el radar mira hacia delante y el coche avanza. Además, la mayoría de las detecciones tienen $|velocidad| < 0.5$ m/s. Por último, los tres montones están unos encima de otros. Las medianas son 0.53 par obstáculos estáticos, 0.12 para vehículos y 0.68 para peatones. Se solapan casi por completo, imposibilitando la distinción.

Conclusión

La presentación de resultados nos da pie a diversas conclusiones que refuerzan todo lo visto a lo largo del Trabajo de Fin de Master.

Primero, una clarificación de los resultados es la que se aprecia en base al algoritmo MAPPO, bajo el esquema de Entrenamiento Centralizado con Ejecución Descentralizada (CTDE). Este, demostró ser capaz de coordinar agentes autónomos en un entorno dinámico con baja adherencia. Los agentes aprendieron políticas equilibradas para seguir rutas, mantener el carril y evitar colisiones. Sin embargo, nos muestra también como bajo ciertas casuísticas se debe trabajar en la limitación de las actualizaciones; al compartir los estados de todos los agentes, algunas veces podía incidir en la reducción de la calidad de la política de uno de los agentes por intentar elevar o asemejar la del otro.

El estudio puso de manifiesto la alta sensibilidad de MAPPO al ruido de observación. Mientras que la búsqueda de soluciones basadas en visión (RGB y segmentación semántica) y radar crudo imposibilitaron la convergencia debido a la dispersión y al ruido que estos sensores introducían en la red, la abstracción mediante Ground Truth resultó clave para estabilizar los gradientes y evaluar el control del sistema. Se puede hacer énfasis aquí, en que el radar validado demuestra que la calidad de la medida no es limitante; pese al ruido y el campo de visión restringido, la política mantiene su rendimiento sin degradación apreciable. Este resultado es positivo, pues nos refuerza la idea de qué la integración con un sistema de percepción que permita etiquetar las entradas, arrojaría resultados muy positivos en un sistema de conducción bajo condiciones meteorológicas de lluvia intensa.

El diseño de la función de recompensa logró evitar comportamientos indeseados y facilitó la generalización del modelo a contextos en los que no se había desarrollado, gracias a incentivar el frenado preventivo ante situaciones críticas. El gran salto de calidad conseguido en este modelo, se observa gracias a la aplicación del Curriculum

Learning, entregando a la política las suficientes muestras para que converja de manera adecuada a la evitación de obstáculos.

El trabajo demuestra que la aplicación de MARL en condiciones climatológicas extremas no presenta una gran dificultad en la capacidad de coordinación multi-agente una vez la función de recompensa permite optimizar la política, sino que esta radica en la degradación de la percepción. El análisis de ablación evidenció que la señal de radar cruda genera interferencias debidas a no diferenciar correctamente objetos estáticos y dinámicos, lo que desestabiliza la política si no existe una etapa previa de filtrado.

Aportaciones del Trabajo

- **Arquitectura MARL robusta con CTDE:** Diseño e implementación de un sistema de conducción autónoma coordinada basado en el algoritmo *Multi-Agent PPO* (MAPPO) bajo el paradigma de Entrenamiento Centralizado con Ejecución Descentralizada (*Centralized Training with Decentralized Execution*, CTDE).
- **Sistema de recompensa modulado y aprendizaje por currículo:** Formulación e implementación de una función de recompensa dotada de mecanismos para penalizar la velocidad en situaciones de riesgo *Speed Gate* y limitar las paradas injustificadas *Stop Excuse* combinada con un esquema de *Curriculum Learning* dinámico para la inyección y liberación de vehículos NPCs.
- **Infraestructura de simulación tolerante a fallos e inestabilidades:** Desarrollo de un marco de ejecución automatizado sobre el simulador CARLA dotado de resiliencia operativa, gestionando la memoria GPU mediante vaciados de caché, escribir registros síncronos y ejecutar reconexiones automáticas periódicas para asegurar la continuidad del entrenamiento masivo conservando las mejoras conseguidas hasta la caída.
- **Evaluación estadística rigurosa de eventos infrecuentes:** Establecimiento de un marco de evaluación metodológica basado en intervalos de confianza de Poisson al 95 % para auditar la tasa de llegada a meta de los agentes, permitiendo discriminar de manera estadística las mejoras reales en la política frente a la varianza estocástica inherente al simulador.

Trabajos Futuros

- **Escalabilidad a un número dinámico de agentes ($N > 2$) mediante Redes Neuronales de Grafos (GNN):** Ampliación del espacio de observación hacia un número indeterminado de agentes guiados por RL. La sustitución de las componentes referidas a la distancia entre agentes del vector de estado por una estructura de grafo dinámico (*Graph Neural Networks*), donde los agentes y obstáculos actúan como nodos y sus interacciones como aristas, permitiendo escalar el sistema a escenarios urbanos densos sin modificar la topología interna de las redes neuronales.
- **Percepción visual comprimida mediante VAEs y cobertura periférica de 360°:** Reincorporación de información visual de alta resolución procedente de un conjunto de cámaras perimétricas (360°) integrándolo haciendo uso de un *Variational Autoencoder* (VAE) preentrenado. El VAE codificará las imágenes en un espacio latente compacto de baja dimensión, desacoplando la extracción de características visuales del proceso de optimización del algoritmo de aprendizaje por refuerzo, facilitando el aprendizaje y comprensión del entorno como ya se demuestra en Kendall et al. [21].
- **Mecanismos de atención espacial en entornos complejos e intersecciones:** Incorporación de capas de atención espacial (*Graph Attention Networks* o *Transformers*) para ponderar la relevancia de los obstáculos y agentes circundantes según la intención de ruta y la cercanía geométrica, mejorando la toma de decisiones cooperativa en cruces.
- **Integración con ROS 2:** Migración de la arquitectura de comunicación y control hacia el ecosistema ROS 2 (*Robot Operating System*). La modularización de los agentes como nodos independientes de ROS 2 facilitará el despliegue en tiempo real.

Lista de referencias

- [1] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio López, and Vladlen Koltun. Carla: An open urban driving simulator. *arXiv preprint arXiv:1711.03938*, 2017.
- [2] Yicheng Guo, Jiaqi Liu, Rongjie Yu, Peng Hang, and Jian Sun. Mappo-pis: A multi-agent proximal policy optimization method with prior intent sharing for cavs' cooperative decision-making. *arXiv preprint arXiv:2408.06656*, 2024.
- [3] Joris Dinneweth, Abderrahmane Boubouzoul, René Mandiau, and Stéphane Espié. Multi-agent reinforcement learning for autonomous vehicles: a survey. *Autonomous Intelligent Systems*, 2(1):27, 2022.
- [4] CARLA Team. Carla python api documentation. https://carla.readthedocs.io/en/latest/python_api/. Accessed: 2026.
- [5] Peilin Wang. Autonomous navigation with deep reinforcement learning in carla simulator. Master's thesis, Technische Universität Dresden, 2021.
- [6] Donglin Yang, Zhenfeng Liu, Wentao Jiang, Guohang Yan, Xing Gao, Botian Shi, Si Liu, and Xinyu Cai. Realistic rainy weather simulation for lidars in carla simulator. *arXiv preprint arXiv:2312.12772*, 2023.
- [7] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [8] KDnuggets. Reinforcement learning: Teaching computers to make optimal decisions. <https://www.kdnuggets.com/2023/07/reinforcement-learning-teaching-computers-make-optimal-decisions.html>, 2023. Accedido: 2026.
- [9] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidje-

- land, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [11] Towards Data Science. Reinforcement learning: Part 2 — policy evaluation and improvement, 2018.
- [12] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, 1992.
- [13] Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. *Advances in Neural Information Processing Systems (NIPS)*, 12, 2000.
- [14] Ruiqi Zhang, Jing Hou, Florian Walter, Shangding Gu, Jiayi Guan, Florian Röhrbein, Yali Du, Panpan Cai, Guang Chen, and Alois Knoll. Multi-agent reinforcement learning for autonomous driving: A survey. *arXiv preprint arXiv:2408.09675*, 2024.
- [15] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, pages 1889–1897, 2015.
- [16] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [17] Stefano V Albrecht and Peter Stone. Autonomous agents modelling other agents: A survey. *Artificial Intelligence*, 258:66–95, 2018.
- [18] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of ppo in cooperative multi-agent games. *arXiv preprint arXiv:2103.01955*, 2022.
- [19] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems (NIPS)*, 30, 2017.
- [20] Sven Gronauer and Klaus Diepold. Multi-agent deep reinforcement learning: a survey. *Artificial Intelligence Review*, 55(2):895–943, 2022.
- [21] Alex Kendall, Jeffrey Hawke, David Janz, Przemyslaw Mazur, Daniele Reda, John-Mark Allen, Vinh-Dieu Lam, Alex Bewley, and Amar Shah. Learning to

- drive in a day. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 8248–8254. IEEE, 2019.
- [22] Anil Ozturk, Mustafa Burak Gunel, Resul Dagdanov, Mirac Ekim Vural, Ferhat Yurdakul, Melih Dal, and Nazim Kemal Ure. Investigating value of curriculum reinforcement learning in autonomous driving under diverse road and weather conditions. *arXiv preprint arXiv:2103.07903*, 2021.
- [23] Jayesh K Gupta, Maxim Egorov, and Mykel Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In *Autonomous Agents and Multiagent Systems (AAMAS Workshops)*, pages 66–83. Springer, 2017.
- [24] Ming Zhou, Jun Luo, Julian Vilella, Yaodong Yang, Xiaojian Ruan, Jiaming Cao, Wenhan Zheng, Ismael Samir, et al. Smarts: Scalable multi-agent reinforcement learning training school for autonomous driving. In *Proceedings of the Conference on Robot Learning (CoRL)*, pages 264–285, 2020.
- [25] Lukas M Schmidt, Johanna Brosig, Axel Plinge, Bjoern M Eskofier, and Christopher Mutschler. An introduction to multi-agent reinforcement learning and review of its application to autonomous mobility. *arXiv preprint arXiv:2203.07676*, 2022.
- [26] Yuxiao Chen et al. Safety guaranteed robust multi-agent reinforcement learning with hierarchical control for connected and automated vehicles. *arXiv preprint arXiv:2309.11057*, 2023.
- [27] Christian Dannheim, Christian Iking, Markus Mader, and Peter Sesselmann. Weather identification in automotive applications based on radar and camera data. In *IEEE 17th International Conference on Intelligent Transportation Systems (ITSC)*, pages 882–889. IEEE, 2014.
- [28] Kshitiz Garg and Shree K. Nayar. When does a camera see rain? *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(7):1061–1074, 2005.
- [29] Louis J. Battan. *Radar Observation of the Atmosphere*. University of Chicago Press, 1973.
- [30] Tecnoempleo. Informe sobre el empleo en informática y sector tic. <https://www.tecnoempleo.com/informe-empleo-informatica.php>. Accedido: 2026.
- [31] Glassdoor. Sueldos para el puesto de scrum master en españa, 2026. Accedido en 2026.

- [32] GitHub. Github copilot: Your ai pair programmer. <https://github.com/features/copilot>. Accedido: 2026.
- [33] Jefatura del Estado. Real decreto 1098/2001, de 12 de octubre, por el que se aprueba el reglamento general de la ley de contratos de las administraciones públicas, 2001. Artículos 130 y 131.
- [34] Agencia Tributaria. Tipos impositivos del iva 2026. https://sede.agenciatributaria.gob.es/static_files/Sede/Tema/IVA/IVA_reperc/, 2026. Accedido: 2026.
- [35] España. Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales. <https://www.boe.es/eli/es/lo/2018/12/05/3/con>, Diciembre 2018.
- [36] Unión Europea. Reglamento (ue) 2016/679 del parlamento europeo y del consejo relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX%3A32016R0679>, Abril 2016.
- [37] Unión Europea. Reglamento (ue) 2024/1689 del parlamento europeo y del consejo, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial (reglamento de inteligencia artificial). <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>, Julio 2024.
- [38] España. Real decreto legislativo 1/1996, de 12 de abril, por el que se aprueba el texto refundido de la ley de propiedad intelectual. <https://www.boe.es/buscar/act.php?id=BOE-A-1996-8930>, Abril 1996.
- [39] Jose Javier González. TFM: Multi-agent deep reinforcement learning for autonomous driving. <https://github.com/JoseJavierGonz/TFM>, 2024. Repositorio de GitHub con el código fuente e implementación del proyecto.

Anexo A

Control de versiones

En el marco de desarrollo del Trabajo de Fin de Máster (TFM), se emplea GitHub como servicio de control de versiones para gestionar eficientemente el código fuente y la documentación del proyecto; facilitando el seguimiento, la trazabilidad y la colaboración. Esta herramienta permite un seguimiento del proyecto por parte del responsable o tutor. Además, de ser necesario, GitHub presenta una funcionalidad para desarrolladores que permite editar el proyecto desde un navegador web. Facilitando la implementación de mejoras y la corrección de errores desde cualquier dispositivo con acceso a internet. Casi la totalidad del desarrollo de este proyecto final lo podemos encontrar en el repositorio personal [39], cumpliendo en todo momento, con la normativa y legislación bajo las que se enmarca el proyecto.

Anexo B

Seguimiento de proyecto fin de máster

B.1. Forma de seguimiento

El seguimiento del proyecto se ha realizado mediante una metodología ágil (*Agile/Scrum*), organizando el trabajo de 4 Sprints a lo largo de 15 semanas. La supervisión se ha llevado a cabo mediante reuniones periódicas (una hora cada dos semanas por parte del tutor/Scrum Master) para evaluar el progreso del trabajo e identificar bloqueos técnicos.

Para la gestión de las tareas y el control de versiones se han utilizado las siguientes herramientas:

- **Reuniones (Google Meet):** Sesiones de seguimiento para evaluar el avance de los Sprints, resolver incidencias y validar los resultados de los entrenamientos.
- **Control de versiones (GitHub):** Gestión del código fuente: implementaciones de MAPPO en PyTorch y entornos de Gymnasium, garantizando la trazabilidad de cada cambio.
- **Redacción colaborativa (Overleaf):** Elaboración y revisión de la memoria en LaTeX a través de Overleaf, facilitando la supervisión y corrección del documento por parte del tutor.

B.2. Planificación inicial

La planificación inicial del proyecto se diseñó con el objetivo de realizarlo en 15 semanas lectivas, equivalente a 225 horas para responsable de la implementación, Ingeniero de IA/Robótica, y 7,5 horas para el rol de Scrum Master. Toda esta distribución temporal se estructuró formalmente en la Sección 3.3, quedando reflejado en el diagrama de Gantt de la Figura 3.1.

Se propuso así una progresión secuencial comenzando con el estudio del estado del arte (Sprint 1), siguiendo con el diseño del escenario y la integración de sensores (Sprints 2 y 3), y finalizando con la fase de experimentación y redacción (Sprint 4 y tareas transversales).

B.3. Planificación final

Durante la ejecución real del proyecto, la planificación no se mantuvo estable en cuanto a plazos. Debido a las restricciones técnicas de la plataforma de simulación y a la complejidad del espacio de observaciones, se produjeron reajustes continuos variando así de tiempo entre etapas:

- **Incremento de carga en desarrollo algorítmico (Sprint 3):** La inestabilidad observada primero con sensores visuales (primero cámara RGB y tras variar el método también segmentación semántica) y radar, sumada a fugas de memoria en CARLA, llevó a derivar un esfuerzo adicional para migrar a la arquitectura *Ground Truth* y programar los mecanismos de reconexión automática.
- **Optimización de la fase experimental (Sprint 4):** El tiempo invertido en robustecer el entorno en el Sprint 3 permitió que los entrenamientos finales sobre Town10HD fueran más estables.

En la Tabla B.1 se recoge la comparativa de dedicación entre la estimación inicial y la ejecución real.

Tabla B.1: Comparativa de dedicación por Sprints entre la planificación inicial y la final.

Sprint / Bloque	Horas Estimadas	Horas Reales	Desviación
Sprint 1: Fundamentos y Entorno	50 h	50 h	0 h
Sprint 2: Diseño del Escenario	40 h	40 h	0 h
Sprint 3: Desarrollo Algorítmico y Sensores	45 h	60 h	+15 h
Sprint 4: Experimentación y Eva- luación	60 h	55 h	−5 h
Tareas Transversales (Memoria)	30 h	30 h	0 h
Total	235 h	235 h	0 h

Anexo C

Declaración de uso de Inteligencia Artificial

En el desarrollo del presente Trabajo de Fin de Máster se han utilizado herramientas de Inteligencia Artificial Generativa como apoyo en tareas específicas de programación y redacción, manteniendo en todo momento la responsabilidad sobre el contenido por parte del autor.

Específicamente, la utilización de estas herramientas se puede distribuir como:

- **GitHub Copilot:** Empleado como asistente de programación durante la implementación del código y para explicación matemática de términos necesarios. Colaborando en la estructuración de los modelos de aprendizaje por refuerzo multiagente y la integración con el simulador.
- **Google Gemini:** Utilizado como asistente de redacción para la revisión estilística y ortográfica del texto, la resolución de dudas sobre la sintaxis LaTeX, el formateo de tablas y la estructuración de la bibliografía en formato BibTeX.

Responsabilidad y autoría

El autor del trabajo asume la responsabilidad sobre el planteamiento del problema, la definición de requisitos, el diseño de la solución y de la arquitectura software, las decisiones técnicas adoptadas, el diseño y ejecución de los experimentos, el análisis de los resultados y la elaboración de las conclusiones.

Las herramientas de inteligencia artificial se han utilizado como instrumentos de apoyo y no como sustituto del proceso de diseño, experimentación o evaluación desa-

rollado por el autor. Todas las aportaciones obtenidas mediante estas herramientas han sido revisadas, adaptadas y, cuando ha sido necesario, verificadas antes de su incorporación al trabajo.

El contenido de la memoria, el código desarrollado, los resultados presentados y las conclusiones obtenidas, con independencia de las herramientas empleadas como apoyo durante su elaboración, es responsabilidad del autor.